Engineering Applications of Artificial Intelligence 148 (2025) 110347



Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai





Using Physics-Informed neural networks for solving Navier-Stokes equations in fluid dynamic complex scenarios

Tommaso Botarelli [a], Marco Fanfani [a], Paolo Nesi [a,*], Lorenzo Pinelli [b]

[a] DISIT Lab, Department of Information Engineering, University of Florence, Via S. Marta 3, 50139, Florence, Italy
[b] T-Group, Department of Industrial Engineering, University of Florence, Via S. Marta 3, 50139, Florence, Italy

ARTICLE INFO

Keywords:
Physics-Informed neural networks
Deep learning
Fluid dynamic
Partial differential equations
Navier-Stokes

ABSTRACT

Navier-Stokes equations used to model fluid dynamic processes are fundamental to address several real-world problems related to energy production, aerospace applications, automotive design, industrial process, etc. However, since in most cases they do not admit any analytical solution, numerical simulations are required in industrial contexts to assess fluid dynamic behaviors in specific setups. Computational Fluid Dynamics (CFD) methods, like those using finite volume or element approaches, are exploited to find Navier-Stokes solutions and carry out simulations. However, such methods require expensive hardware resources, relevant computational times, and manual efforts for the definition of dense meshes on which equations are evaluated iteratively for each time step of the simulation. Physics-Informed Neural Networks (PINNs), which are deep neural networks where physical laws are directly embedded into the training process, offer a promising approach for solving Navier-Stokes equations, thus alleviating hardware and time requirements. PINNs bypass some CFD limitations by using neural networks to produce solutions based on governing equations, thus reducing the need for large datasets, dense meshing, and iterative estimation over time. This paper evaluates the application of PINNs in near real-world scenarios, while considering various geometries. The study focuses on the achieved accuracy, by comparing PINN estimates with CFD solutions obtained via OpenFOAM, and the required training times; this includes evaluating different neural network architectures, activation functions, and numbers of sampling points. Additionally, several training strategies such as fine-tuning, multi-resolution learning, and parametrized training are proposed to enhance efficiency and obtain speed up. Results demonstrate that PINNs can achieve comparable accuracy to CFD methods (with a velocity magnitude mean absolute error inferior to 10^{-2}) and significantly reduce computational costs. Our findings demonstrated that with appropriate training techniques PINNs can be effectively used in industrial applications requiring rapid and accurate fluid dynamic simulations, thus paving the way for their broader adoption in practical engineering problems.

1. Introduction

Modelling Fluid Dynamic (FD) problems is fundamental in several disciplines like astrophysics, chemistry, environmental sciences, civil, biomedical, and mechanical engineering. The goal is to simulate complex processes to make predictions on specific scenarios, while avoiding the necessity of conducting expensive real-world experiments in the design phase. FD problems are modeled by the Navier-Stokes (N-S) equations, a system of partial differential equations (PDEs) describing the momentum and energy balances, together with the conservation of mass. The application of N-S equation to engineering studies can range from external aerodynamic problems (aircraft fuselage and wing, racing cars, wind turbines, etc.) to internal flow problems (aircraft engines, piping, injectors, autoclave curing, etc.), as well as fluid mixture investigations (steam condensing) and reacting flows (combustion systems). Except for some simplified specific problems, the general form of N-S equations does not admit known analytical solutions and may be solved by means of numerical approaches. Computational Fluid Dynamic (CFD) methods based on finite elements/volumes/difference (Zienkiewicz et al., 2005; Szabó and Babuška, 2021) have been widely used for solving N-S equations to estimate flow velocity and pressure fields given specific Boundary and Initial Conditions (BC and IC). However, such approaches have strong limitations, since they require considerable time for both their instantiation and execution (Pinelli

* Corresponding author.
E-mail address: paolo.nesi@unifi.it (P. Nesi).

https://doi.org/10.1016/j.engappai.2025.110347
Received 11 August 2024; Received in revised form 18 January 2025; Accepted 15 February 2025
Available online 7 March 2025
0952-1976/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

et al., 2022). CFD methods need spatial and temporal domain discretization (i.e., the definition of meshes or grids) on the basis of which approximate solutions exploiting discrete models are computed. In the case of very complex scenarios of the fluid domain, the mesh generation may not be straightforward and is usually time-consuming, requiring manual efforts from expert engineers. Moreover, high fidelity CFD simulations, based on Large Eddy Simulations (LES) or better Direct Numerical Simulations (DNS), are very computationally expensive, which means long execution times and expensive hardware like clusters of several up-to-date CPUs. For these reasons, they are unfeasible for many real-world applications where the availability of adequate hardware is limited, and the use of highly specialized human resources is extremely costly. Moreover, in Industry 4.0 contexts where fast simulations and quasi real time applications are required (Ferreira et al., 2020), execution times of CFD simulation are impractical.

Machine Learning (ML) approaches have been proposed to address FD problems (Linse and Stengel, 1993; Faller and Schreck, 1997), and with the increasing diffusion of Deep Learning (DL) (Tamil Thendral et al., 2022; Cao et al., 2024a; Cao et al., 2024b) a renewed attention has been devoted to this topic in the last decade. By using physical measurements gathered during manufacturing process or numerical data obtained through CFD simulations, ML models can be trained to predict PDE governed processes (Parish and Duraisamy, 2016; Wang et al., 2017; Barzegar Gerdroodbary, 2020). Nevertheless, the difficulty of acquiring sufficient data for training and the specificity of some processes represents a strong limitation to actual usage of such approaches. More interesting are the Physics-Informed Neural Networks (PINNs) (Raissi et al., 2019), where the PDE solution is obtained by directly enforcing the equations into the training loss function of the neural network, with Automatic Differentiation (Baydin et al., 2018) to be exploited to compute the derivatives. By minimizing the loss on a set of points, the network is trained and converges to produce outputs that should be equal to the PDE model solution. Moreover, IC and BC can be included in the loss function as additional terms. Even if it is possible to include numerical data in the PINN training process, such data needs on training are strongly reduced or better not required. Indeed, PINNs can potentially alleviate most limits of traditional CFD by providing more economic and fast solutions for FD problems. By exploiting consumer hardware as a single workstation equipped with a modern GPU, costs can be reduced; furthermore, since PINNs do not require complex finite difference computations and exploit highly optimized deep-learning framework, simulations can be achieved with reduced times.

However, PINNs are relatively new techniques and solutions. Works on PINNs have emerged over the last few years (Lawal et al., 2022). Most of them have focused on proposing improvements on benchmark test cases, such as: evaluating soft or hard BC and IC (Sun et al., 2020), training on multiple consecutive time windows (Meng et al., 2020), or providing general best-practices (Wang et al., 2023a); there has been no attempt of actually evaluating the applicability of PINNs in real scenarios. Most real-world applications have scenarios with specific setups, for example, ventilation in an open pit (Flores et al., 2013), flow past multiple rectangles (Ghamlouch et al., 2017; Fezai et al., 2020) or diamond-shaped obstacles (Djeddi et al., 2013). Moreover, there is the need to assess the correlation between the result accuracy and the training time, as well as the possibility to employ transfer learning or fine-tuning techniques to reduce the time required for the training, when passing from one scenario to another.

In this paper, we present results about the usage of PINN in close to real-world scenarios for the resolution of Navier-Stokes equations. Complex domains with different geometries have been used to show performance of different neural architectures, activation functions, layer dimensions, network depths, while considering the impact of the number of sampling/collocation points. All results are discussed by taking into account both achieved errors and required training times. Moreover, we have proposed alternative learning strategies that can be exploited to reduce training times and still obtain valid estimates with low errors.

2. Related works, paper aims and organization

Since the seminal work of Raissi et al. (2019), relevant attention has been given to PINNs, as a novel approach to solve PDE problems. In (Raissi et al., 2019), examples to show the applicability of PINNs included the resolution of non-linear Schrödinger and Allen–Cahn equations in data-free setups. Some data from a spectral solver are instead used together with the physical losses to address the Navier–-Stokes (applied to the prototype problem of incompressible flow past a circular cylinder), Korteweg–de Vries, and Burger's equations. In (Sun et al., 2020), a physics-constrained deep learning approach in a data free setup was proposed to solve N-S equations, with 2D pipes, stenotic, and aneurysmal flows as case studies. Even if they were inspired by biomedical problems, the setups/scenarios were somewhat simple to demonstrate the applicability of PINNs on real cases. Burger's and 2D nonlinear diffusion–reaction equations were addressed in (Meng et al., 2020) and the paper provided an analysis on the usage of multiple time windows to reduce total training time, however without giving clear indications about the relation among the number of epochs, the number of computation grid points, and the obtained error in field flow estimation. In (Xiang et al., 2021), solutions of N-S equations are searched to solve 2D steady Kovasznay's flow passing on a circular cylinder, and as a Beltrami's flow. In (Jin et al., 2021), the assessment of PINNs in solving N-S equation incompressible flow on benchmark test cases (steady Kovasznay's flow, flow passing in a circular cylinder, and unsteady Beltrami's flow) studying the impact of the number of grid sampling points, different architectures, optimizers, and loss weights were proposed. A simple test on transfer learning was provided. However, no clear indications about training times have been reported and applications on different use cases closer to real scenarios were not provided. In (Eivazi et al., 2022), the authors evaluated PINNs for solving Falkner–Skan boundary layer and 4 turbulent-flow cases. To show how difficult PINN training can be in a data-free context, in (Chuang and Barba, 2022), some failure cases have been reported, by considering the Taylor-Green vortex problem and the flow passing into a circular cylinder. Similar difficulties have been highlighted in (Wang et al., 2023a), by providing tips and best practices about architectures and training strategies to improve PINN results in solving PDEs for a number of equations and models such as: Allen-Cahn and Kuramoto–Sivashinsky equations, lid-driven cavity flow, N-S flow in a torus and around a cylinder. The flow passing into a circular cylinder has been addressed in (Hu and McDaniel, 2023), by considering an elliptical particle in (Xu et al., 2023), in which they evaluated PINNs with limited data (Naderibeni et al., 2024). The Taylor–Green vortex test case has been addressed in (De Ryck et al., 2024) siding PDE related losses of a PINN with data coming from CFD solvers.

In Table 1, a summary of FD problems addressed in recent literature by PINN is provided. To summarize, most papers on PINNs were focused on benchmark problems of PDE and FD. Moreover, several solutions exploited numerical data, thus spoiling the assessment of PINNs as substitutes of classical CFD solvers, and very few works offered detailed analysis on execution times with respect to estimation accuracies, by using transfer learning, fine-tuning, or other strategies.

2.1. Paper aims and organization

In this paper, we have reported the results of the application of PINN on computing flows for cases close to real-world scenarios, while taking into account both estimation errors and execution times. The solutions produced with PINN have been compared against CFD solutions carried out by using OpenFOAM, which is a well-known CFD solver suite (OpenFoam soft.) exploiting the finite volume method (FVM) and validated by the authors on turbine and acoustic liner application (Giaccherini et al., 2023).

Table 1
summary of FD N-S problems addressed with PINNs in the literature. Symbols: Y = yes, N = no, L = limited; C=CFD simulation, A = analytic solution. Most state-of-the-art related works focused on benchmark problems with simple geometries and some of them used data in training, partially spoiling the potentiality of PINN. Moreover, few of them presented analysis for the transient phase of the F-D processes, evaluated the trade-off between estimation accuracy and training times, carried out an assessment of neural architectures, or proposed training strategies. Differently the proposed study addresses all these analyses using as test cases complex geometry setups closer to real-world applications.

Paper	Channel Geometry	Piece Geometry	Equation/ Problem	Validation	Time dependency	Transient analysis	Data free	Time vs errors	Test architectures	Test activations	Test collocation points	Comparison of different training strategies
Raissi et al. (2019)	2D Pipe	Circular cylinder	Navier–Stokes	C	Y	N	N	N	N	N	N	None
Sun et al. (2020)	2D Pipe 2D stenosis	No piece	Poiseuille flow	C	N	N	Y	Y	Y	Y	Y	Hard BC
Meng et al. (2020)	1D Closed Pipe	No piece	Burgers equations	C	Y	Y	Y	Y	N	N	N	Hierarchical time windows
Xiang et al. (2021)	2D Rectangular periodic BC	No piece	Kovasznay flow	A	Y	N	Y	N	N	N	N	Loss balancing
	2D Pipe	Circular cylinder	Navier–Stokes	C	Y	N	N					
	3D Box periodic BC	No piece	Beltrami flow	A	Y	N	Y					
Jin et al. (2021)	2D Rectangular periodic BC	No piece	Kovasznay flow	A	Y	N	Y	L	Y	Y	Y	Fine-tuning
	2D Pipe	Circular cylinder	Navier–Stokes	C	Y	N	Y					
	3D Box periodic BC	No piece	Beltrami flow	A	Y	Y	Y					
	3D Channel	No piece	Navier–Stokes	C	Y	Y	Y					
Eivazi et al. (2022)	2D Pipe	Falkner–Skan boundary layer	Navier–Stokes	C	N	N	N	N	N	N	N	None
	2D Pipe	ZPG turbulent boundary layer	Navier–Stokes	C	N	N	N					
	2D Pipe	APG turbulent boundary layer	Navier–Stokes	C	N	N	N					
	2D Pipe	NACA4412 airfoil	Navier–Stokes	C	N	N	N					
	2D Pipe	Periodic hill	Navier–Stokes	C	N	N	N					
Chuang and Barba (2022)	2D Box periodic BC	No piece	Taylor–Green vortex	A	Y	Y	Y	Y	N	N	N	None
	2D Pipe	Circular cylinder	Navier–Stokes	C	Y	Y	Y					
Wang et al. (2023)	2D Pipe	Circular cylinder	Stokes flow	A	N	N	Y	Y	Y	N	N	Curriculum training
	2D square cavity	None	Lid-driven cavity flow	C	N	N	Y					
	2D Torus	None	Navier–Stokes	C	Y	N	Y					
	2D Pipe	Circular cylinder	Navier–Stokes	C	Y	N	Y					
Hu and McDaniel (2023)	2D Pipe	Circular cylinder	Navier–Stokes	C	N	N	Y	N	N	N	N	None
	2D Pipe	Elliptical particle	Navier–Stokes	C	N	N	Y					
Xu et al. (2023)	2D Pipe	Circular cylinder	Navier–Stokes	C	Y	Y	N	N	N	N	N	Effects of external (CFD) data in training
Naderibeni et al. (2024)	2D Pipe	Circular cylinder	Navier–Stokes	C	Y	Y	N	N	N	N	N	None
De Ryck et al. (2024)	2D Box periodic BC	No piece	Taylor–Green vortex	A	Y	N	N	N	Y	N	Y	None
Proposed	2D Pipe, with small inlet/outlet	Single rectangular box	Navier–Stokes	C	Y	Y	Y	Y	Y	Y	Y	Fine-tuning Multiresolution learning
	2D Pipe, with small inlet/outlet	Circular cylinder	Navier–Stokes	C	Y	Y	Y					Parametrized learning
	2D Pipe, with small inlet/outlet	3 rectangular boxes	Navier–Stokes	C	Y	Y	Y					

The main contributions of the paper are.

- **Complex setups/scenarios.** As highlighted above and summarized in Table 1, most of the work on PINN has considered benchmark setups very far from real world application. On the contrary, we have considered a 2D closed rectangular domain, which can be regarded as pipe segment or tunnel, with small apertures for both inlet and outlet flows, with the aim to get closer to real-world applications. Such a setup/scenario is inspired by industrial autoclaves in which controlled flows are used to transport the heating energy required to cure composite materials (Fernlund et al., 2018; Clyne and Hull, 2019): in fact, an accurate understanding of the time varying flow field with respect to the immersed piece is fundamental to obtain correct curing processes. Therefore, the first contribution of this paper is the assessment of PINN performance on more complex setups, not yet addressed in the literature.
- **Evaluation of neural architectures (accuracy and training times).** During PINN training, a self-supervised approach has been exploited relying solely on physical losses without any data. A multiple time-window technique has been used to optimize the results. Tests on different neural architectures, number of neurons per layer (i.e., layer width), number of layers (i.e., network depth), and activation functions have been carried out to assess PINN performance in producing flow field results. Moreover, particular attention has been devoted to training times, which is something not sufficiently considered in literature. In this context, specific tests to observe the impact of different numbers of sampling points have been conducted to show the achievable trade-off between times and accuracy.
- **Multiple piece/obstacle geometries.** Case studies have been addressed always considering multiple geometries for the pieces placed inside the channel, an aspect that is different from the geometries used in state-of-the-art works: the flow field around a single rectangular piece has been initially studied, then spherical and multiple rectangular pieces have been analyzed.
- **Training strategies.** Results for a set of alternative training strategies aimed at the reduction of training times have been issued: (A) techniques of fine tuning applied to the multiple time-windows and to transfer the trained model in different setups, a critical aspect for massive usage of the solution in differentiated productions, (B) a newly devised multi resolution training approach, and (C) a parametrized training to compute the solving model once and reuse it in execution with a large range of difference pieces.

The paper is organized as follows and it is described in Fig. 1. In Section 3, Navier-Stokes PDE equations are briefly described, with scenario configuration being defined. Section 4 presents the CFD method used to produce the PDE results employed as reference in the following analysis. In Section 5, the proposed PINN based approach for PDE solution is described. Section 6 presents the experimental results that have been carried out to identify the best configuration on the basis of different architectures, multiple activation functions, various numbers of neurons per layer, and different number of layers. A rectangular box-like obstacle has been used as a first case. Then novel trainings have been conducted on different obstacle geometries to confirm the validity of the defined setup. An evaluation of the impact of using a different number of points to assess the trade-off between errors and training times is reported at the end of the section. Then, Section 7 describes possible solutions devised to reduce training times by exploiting alternative training strategies, such as: (A) fine-tuning, (B) multi-resolution learning, and (C) parametrized learning. Finally, conclusions are drawn in Section 8.

3. Modeling PDEs and problem definition

This section provides a brief introduction on N-S equations and the description of the test cases closer to real scenarios considering different pieces inside a channel with small inlet/outlet apertures (not a simple pipe). Then, details on both the CFD solver used to produce validation data, and on the PINN architectures and implemented characteristics are reported in the following sections.

According to N-S equation, the dynamic of a fluid is governed by mass, momentum and energy conservation. For non-reacting incompressible flows, mass and momentum equations are decupled from energy equation. In the tensorial notation with Einstein rule, equations have the following form:

$$\frac{\partial u_i}{\partial x_i} = 0 \tag{1}$$

$$\frac{\partial u_j}{\partial t} + \frac{\partial u_i u_j}{\partial x_i} = -\frac{1}{\rho}\frac{\partial p}{\partial x_j} + \frac{1}{\rho}\frac{\partial \tau_{ij}}{\partial x_i} \tag{2}$$

where: u_i is the ith component of the velocity, t is the time, ρ the fluid density, p the pressure, and τ_{ij} the fluid viscous stress tensor that for Newtonian flow is given by

$$\tau_{ij} = \mu\left[\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i}\right] \tag{3}$$

where μ is the dynamic viscosity of the fluid.

In this paper, we have addressed the problem of solving velocity and pressure fields evolution in a closed box-channel with small apertures $\mathbb{C}$ (shown in Fig. 2) with FVM and PINN approaches. The channel length and height go from -0.5 to 0.5 and from -0.25 to 0.25, respectively, according to a normalized shape dimension. Inlet (Γ_{inlet}) and outlet

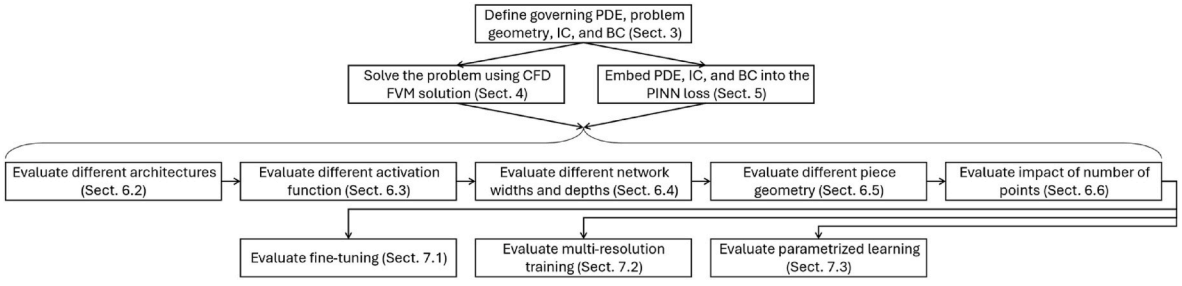


Fig. 1. Block diagram of experimental activity and paper organization. After defining the governing PDEs, the problem geometry, and the initial and boundary conditions (IC and BC) in Section 3, the CDF FMV solution used as benchmark comparison and the PINN solution with the implemented loss function were described in Sections 4 and 5, respectively. Then, Section 6 was devoted to assessing the best neural configuration by means of a sequence of tests considering different architectures (Section 6.2), different activation functions (Section 6.3), different network widths and depths (Section 6.4), different piece geometries (Section 6.5), and different numbers of points (Section 6.6). With the outcomes of Section 6, in Section 7 different training strategies aimed at reducing training times were evaluated: fine-tuning (Section 7.1), multi-resolution training (Section 7.2), and parametrized learning (Section 7.3). For all the tests, accuracies in terms of mean average errors between the velocity and pressure estimates outputted by the CFD/FVM method and by PINN, and PINN training times were reported.

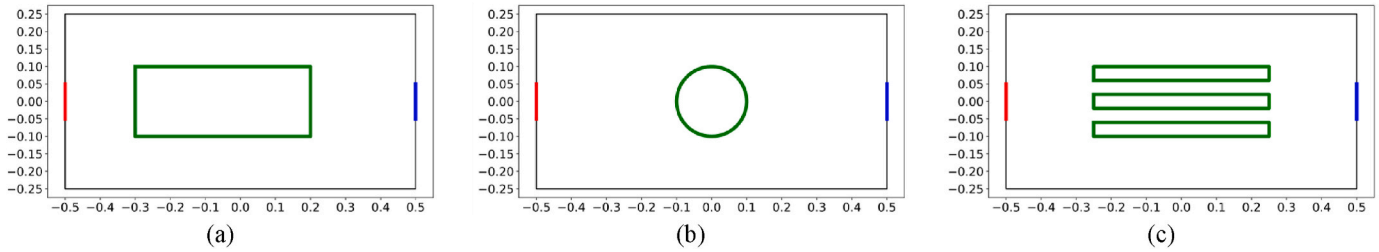


Fig. 2. Graphical representations of the geometrical setups used. The channel is defined as a closed box with small apertures for the inlet and the outlet (indicated in red and blue, respectively). Different pieces have been placed inside the channel and are represented with green borders: (a) a single rectangular box; (b) a circular cylinder; and (c) three small boxes. Dimensions are in meters, normalized. (Best viewed in color). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

(Γ_{outlet}), represented with red and blue lines in Fig. 2, are centered around the longitudinal axis with a width of 0.1 (from −0.05 to 0.05). In this container for fluid flow, three different kinds of piece have been considered, represented in green in Fig. 2: a single solid box (see Fig. 2 (a)) with width of 0.5 (from −0.3 to 0.2) and height of 0.2 (from −0.1 to 0.1); a circular cylinder (see Fig. 2(b)) centered in (0,0) and with radius of 0.1; and three small solid boxes (see Fig. 2(c)) with each box having width 0.5 and height 0.04 spaced on the vertical axis by 0.04.

In order to solve N-S equations (1) and (2), IC and BC have been specified as follows for the whole temporal domain from 0 to 3, addressing the transient fluid flow from the instants when the fluid enters from the inlet. On the inlet, where the flow enters the channel, the flow velocity is instantaneously set to

$$\left(u_x(x,y,t), u_y(x,y,t)\right) = (1,0) \quad \forall\, t \in [0,3] \wedge \forall\, (x,y) \in \Gamma_{inlet} \tag{4}$$

where: $u_x(x,y,t)$ is the longitudinal component, and $u_y(x,y,t)$ is the vertical component of the velocity. For the outlet, where the flow exits the channel, the longitudinal velocity u has been set as

$$u_x(x,y,t) = 1 \quad \forall\, t \in [0,3] \wedge \forall\, (x,y) \in \Gamma_{outlet} \tag{5}$$

Note that no constraints on pressure have been used, leaving the solution to manifest different pressions conditions along with the fluid flow solution. For all the wall surfaces, both channel (Γ_{chan}) and piece(s) (Γ_{piece}), *no slip condition* has been defined – i.e., solid walls that do not admit flow penetration:

$$\left(u_x(x,y,t), u_y(x,y,t)\right) = (0,0) \quad \forall\, t \in [0,3] \wedge \forall\, (x,y) \in \Gamma_{chan}, \Gamma_{piece} \tag{6}$$

The PINN formulation may be used to solve the nondimensional/normalized form of the incompressible N-S equations, that can be obtained by defining a reference length (L) and velocity vector ($U = [u_x, u_y]$). By using these two reference values, quantities of equations (1) and (2) can be made nondimensional by setting:

$$\acute{x} = \frac{x}{L} \quad \acute{u} = \frac{u}{U} \quad \acute{p} = \frac{p}{\rho U^2} \quad \acute{t} = \frac{U}{L}t \tag{7}$$

and thus, the N-S equations without dimensions can be rewritten as:

$$\frac{\partial \acute{u}_i}{\partial \acute{x}_i} = 0 \tag{8}$$

$$\frac{\partial \acute{u}_j}{\partial \acute{t}} + \frac{\partial \acute{u}_i \acute{u}_j}{\partial \acute{x}_i} = -\frac{\partial \acute{p}}{\partial \acute{x}_j} + \frac{1}{Re}\frac{\partial^2 \acute{u}_j}{\partial \acute{x}_i \acute{x}_i} \tag{9}$$

where: ν is the kinematic viscosity, and Re is the Reynolds number defined as:

$$Re = \frac{\rho U L}{\mu} = \frac{UL}{\nu} \tag{10}$$

As to the problem under investigation, the reference length (L) is equal to the box x dimension ($1\ m$). The kinematic viscosity is set to $1.51 \times 10^{-5}\ m^2/s$ as the fluid is air, whereas the reference velocity (U) can be directly derived from the Re number selected for simulations. Note that, in the FVM approach, the equation nondimensionalization is internally performed by the solver returning simulation results in a dimensional form. On such grounds, for a direct comparison against PINN nondimensional results, the scaling factors reported in equation (7) are applied.

4. Solving N-S PDE via finite volume method (FVM)

4.1. Background theory of FVM

FVM basically consists of four main key-steps, discussed in the following from (i) to (iv). **(i) Spatial discretization**: the fluid domain must be discretized into a finite number of volumes (or cells) usually organized according to structured grid-like mesh. **(ii) Integral form of PDEs**: the N-S equations are transformed from their differential form to an integral representation, more suitable to be solved by the FVM. For example, the continuity equation for mass conservation of incompressible flows – equation (1) – $\nabla \boldsymbol{u} = 0$, where $\boldsymbol{u} = [u_x, u_y]^\top$, is rewritten as

$$\int_{V_i} \nabla \boldsymbol{u}\, dV = 0 \tag{10}$$

Where: V_i is a control volume. Using the Divergence Theorem, it is possible to convert the volume integral to surface integral, that is

$$\int_{V_i} \nabla \boldsymbol{u}\, dV = \oint_{\partial V_i} \boldsymbol{u}\boldsymbol{n}\, dA \tag{11}$$

where: ∂V_i is the surface of the control volume, $\boldsymbol{n}$ is the outward pointing normal vector on this surface, and A is the area of the surface of the control volume. In practice, equation (11) states that the net mass flow in and out the control volume must be zero for incompressible flow. A similar formulation can be obtained for the momentum and energy equation (2). Applying the integral form of these equations ensures the mass, momentum, and energy conservation in each control volume. Integral form requires also a discretization of the equations, in order to be numerically solved in the following steps. For example, equation (11) can be discretized with the following approximation

$$\oint_{\partial V_i} \boldsymbol{u}\boldsymbol{n}\, dA \approx \sum_f \boldsymbol{u}^f \boldsymbol{n}^f \Delta A_f = 0 \tag{12}$$

where: the summation works on all the volume faces f, while $\boldsymbol{u}^f$ is the velocity at the face, $\boldsymbol{n}^f$ is the normal, and ΔA_f the face area. In a 2D regular grid, with cell size $\Delta x \times \Delta y$, equation (12) becomes

$$\left(u_x^e - u_x^w\right)\Delta y + \left(u_y^n - u_y^s\right)\Delta x = 0 \tag{13}$$

where: e, w, n, s indicate the east, west, north, and south cell faces. **(iii) Flux approximation**: using numerical approximations (Hirsch, 2007),

fluxes at the boundaries (faces) of each control volume are estimated through interpolation of neighboring cells. For example, considering the Central Differencing Scheme, we get $u_x^e = (u_x^P + u_x^E)/2$, $u_x^w = (u_x^W + u_x^P)/2$, $u_y^n = \left(u_y^P + u_y^N\right)/2$, and $u_y^s = \left(u_y^S + u_y^P\right)/2$, where P is the cell centroid, and E, W, N, and S are the east, west, north, and south neighboring cells. By substituting these values in (13), we finally obtain

$$\frac{(u_x^E - u_x^W)}{2}\Delta y + \frac{\left(u_y^N - u_y^S\right)}{2}\Delta x = 0 \tag{14}$$

that is a finite difference in the spatial domain. A similar derivation can be applied to the momentum and energy equations including both spatial and temporal domains and considering also second order derivatives (see for example the second order backward scheme applied in OpenFoam, described in the following). **(iv) Equation resolution**: a system of algebraic equations is defined gathering any contributions from all the cells to obtain velocity and pressure fields.

4.2. FVM implementation details

In this work, we considered three different piece geometries, namely single-box, circular cylinder, and three-boxes (see Fig. 2), and thus different spatial domain discretization are required according to such different piece geometry. The defined grid meshes for FVM solution are shown in Fig. 3. As to the single-box and the three-boxes, meshes are regular rectangular grids. On the other hand, a more complex grid has been defined for the circular cylinder test case, in order to accurately cover the areas around the rounded shape. More in detail, the grid meshes for finite element computation have been generated with the *blockMesh* utility of the OpenFOAM suite and consist of body-fitted unstructured multi-block grids with wall clustering for boundary layer resolution ($y^+ \approx 1$). The mesh generation process needed a parametric definition of the domain performed by the user, and a successive mesh tuning to achieve better mesh quality standards in term of skewness and non-orthogonality. Meshes are three-dimensional with one cell along the third dimension to solve a 2D problem. About 30000 hexahedral elements have been defined for each test case.

In order to compute the solution, addressing the fourth step of the FVM pipeline, the *icoFoam* solver has been selected. *icoFoam* solves the incompressible laminar N-S equations by using the PISO algorithm (Pressure-Implicit with Splitting Operators), developed for non-iterative computation of unsteady flows. The code is inherently transient and can handle mesh non-orthogonality by means of non-orthogonality iterations. PISO involves predictor and corrector steps to satisfy mass conservation: the number of PISO corrections can be also set by the user. Concerning problems under investigation, 1 non-orthogonal iteration and 2 PISO corrections are selected. For time discretization a second order backward scheme has been selected, therefore the temporal derivative of equation (2) is computed as follows

$$\frac{\partial u_j}{\partial t} \approx \frac{3u_j - 4u_j^0 + u_j^{00}}{2\Delta t} \tag{15}$$

where: the superscript 0 stands for the value at the previous time, and 00 is the value two time-steps ago. Convective terms are discretized with a second order upwind scheme (Hirsch, 2007), while viscous (diffusive) terms are linear corrected. The time step for the transient evolution has been set to 5e-4s in order to maintain a CFL (Courant-Friedrichs-Lewy) number around 1. Inlet and outlet conditions impose constant velocities on the respective patches, while no slip conditions are selected for external box and obstacle surface.

Each simulation has been run in a parallel CPU-based environment and took around 6 h (8 cores of a CPU Intel Core i9-11900K at 3.5 GHz) to perform 30000 cells on 200s duration of simulation with time-step (5e-4s) used to keep the algorithm stable.

5. Using PINN for CFD/FVM simulation/solution

5.1. Background theory

Neural networks (NNs) can be regarded as a sort of function approximators and our objective is to train a NN $\mathcal{N}$ taking as input spatial and temporal coordinates and as output the respective velocity and pressure, i.e., $\mathcal{N}(x,y,t;\theta) = (u_x,u_y,p)$, where θ indicates the network weights and biases. While this kind of problem could be addressed by using classical NN and a huge amount of data in training, the idea behind PINNs is to directly exploit the equations governing the physical phenomena to train the network, without relying on training data. This can be achieved by introducing in the NN training the so-called residual network that starting from the NN outputs (velocity and pressure) applies Automatic Differentiation (AD) to compute first and second order derivatives and obtain the PDE residual. Differently from the numerical differentiation used in FVM (described in Section 4), AD computes derivatives by breaking down functions in simple operations and exploiting the chain-rule. This is possible since in the computational graph defined by the NN, any intermediate values obtained during the forward passing are recorded and used. Once the (u_x, u_y, p) outputs are available, AD is used to compute the derivatives of the PDEs with respect to the output layer. Then, such derivatives are propagated back through other layers, until the first layer is reached and the PDE derivatives with respect to the inputs (x, y, t) are obtained. Derivatives are used to compute any physical residuals of the network loss function, also including IC and BC losses. In addition, other terms related to external data coming from simulations or real computations could be used. Please note that in this work external data are not used in training, thus putting the solver in the worst possible case.

Compared to numerical derivative computations, AD is more accurate, since it should not suffer from truncation or round-off errors. Moreover, the PINN approach compared to FVM avoids the need to define huge dense mesh grids and related computation at each iteration in space and time, thus reducing time and complexity for the experimental setup preparation. When all the loss components are ready, the derivatives of the loss function are computed exploiting the AD, performing the well-known backpropagation based on which the network

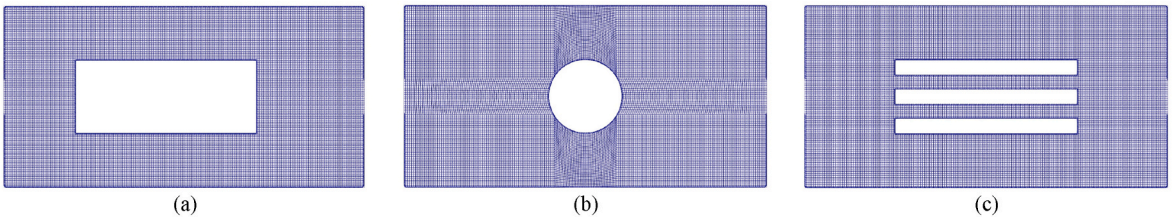


Fig. 3. Computational meshes for FVM. Note that the BCs impact on all the four steps: they are used to define the physical boundaries of the problem domain in step (i), they influence the definition of the integral equation in step (ii), e.g., by specifying no-slip conditions on the faces of control volumes placed near the channel or piece border, and they are used in step (iii) to compute the fluxes in control volumes on domain borders. Both BC and IC are used in step (iv): BC are continuously applied to correctly constrain the solution; IC are used to initialize the system variables and kick-start the iterative process.

weights and bias optimization are carried out.

5.2. Implementation details for N-S equation resolution

In the case of N-S equations ((1) and (2) or (8) and (9) for their a-dimensional representation), the right parts of equations are moved on the left part to have all equations equals to zero. Then, with the forward pass of $\mathcal{N}$, output estimates of velocity and pressure for a set of points $c \in \mathbb{C}$ are obtained, i.e., $\left\{u_x^c, u_y^c, p^c\right\}$. AD is used to compute the partial derivatives (e.g., $\partial u_x/\partial x$ and $\partial u_y/\partial y$ for the continuity equation (1)) to evaluate the PDE residuals. Briefly, the system evaluates how much the derivatives of output velocity and pressure estimates combined according to (1) and (2) (i.e., (8) and (9)) differ from zero, thus producing an indication on how close the net is to correctly estimate velocity and pressure. Therefore, the first terms in the loss function $\mathcal{L}$ are the PDE residuals for the N-S momentum ($\mathcal{L}_M^x, \mathcal{L}_M^y$) and continuity ($\mathcal{L}_C$) equations. In addition, IC and BC are also enforced into the loss function to constrain the net in learning the correct solution. IC and BC are evaluated on specific points lying on the inlet (Γ_{inlet}), outlet (Γ_{outlet}), channel (Γ_{chan}) and piece (Γ_{piece}) walls, i.e.:

$$\mathcal{L}_{in}^x = \frac{1}{N_i}\sum_{i=1}^{N_i} |u_x(i,t)-1|, \mathcal{L}_{in}^y = \frac{1}{N_i}\sum_{i=1}^{N_i} |u_y(i,t)| \quad \forall\, i \in \Gamma_{inlet} \wedge t \in [0,3] \tag{16}$$

$$\mathcal{L}_{out} = \frac{1}{N_o}\sum_{o=1}^{N_o} |u_x(i,t)-1| \quad \forall\, o \in \Gamma_{outlet} \wedge t \in [0,3] \tag{17}$$

$$\mathcal{L}_{cw}^x = \frac{1}{N_{c_w}}\sum_{c_w=1}^{N_{c_w}} |u_x(c_w,t)|, \mathcal{L}_{cw}^y = \frac{1}{N_{c_w}}\sum_{c_w=1}^{N_{c_w}} |u_y(c_w,t)| \quad \forall\, c_w \in \Gamma_{chan} \wedge t \in [0,3] \tag{18}$$

$$\mathcal{L}_{pw}^x = \frac{1}{N_{p_w}}\sum_{p_w=1}^{N_{p_w}} |u_x(p_w,t)|, \mathcal{L}_{pw}^y = \frac{1}{N_{p_w}}\sum_{p_w=1}^{N_{p_w}} |u_y(p_w,t)| \quad \forall\, p_w \in \Gamma_{piece} \wedge t \in [0,3] \tag{19}$$

where: $\mathcal{L}_{in}^x, \mathcal{L}_{in}^y, \mathcal{L}_{out}$ are related to the IC on inlet and outlet, $\mathcal{L}_{cw}^x, \mathcal{L}_{cw}^y$ and $\mathcal{L}_{pw}^x, \mathcal{L}_{pw}^y$ to the BC on the channel and piece walls, and N_i, N_o, N_{c_w}, and N_{p_w} indicate the total number of points used in the inlet, outlet, channel and piece wall respectively. The final multi-objective loss function $\mathcal{L}$ is defined as follows:

$$\mathcal{L} = w_M^x \mathcal{L}_M^x + w_M^y \mathcal{L}_M^y + w_C \mathcal{L}_C + w_{in}^x \mathcal{L}_{in}^x + w_{in}^y \mathcal{L}_{in}^y + w_{out} \mathcal{L}_{out} + w_{cw}^x \mathcal{L}_{cw}^x + w_{cw}^y \mathcal{L}_{cw}^y + w_{pw}^x \mathcal{L}_{pw}^x + w_{pw}^y \mathcal{L}_{pw}^y \tag{20}$$

where: w_M^x, w_M^y, w_C, w_{in}^x, w_{in}^y, w_{out}, w_{cw}^x, w_{cw}^y and w_{pw}^x, w_{pw}^y, are the weights used to obtain $\mathcal{L}$ by linear scalarization. Higher the weight, greater is the impact of the specific loss/constraint. However, in order to obtain valid results all the constraints must be considered, and giving to some of the sub-losses a higher importance/weight lead to unsatisfactory outcomes. In addition, the sub-losses can have different scales that, if not considered, could result in an unbalanced loss. Moreover, the losses can have different scales, not easy to know before training, that if not considered could lead to an unbalanced loss. Therefore, manual tuning of the weights is very difficult. For this reason, we have used the ReLoBraLo (Relative Loss Balancing with Random Lookback) balancing schema (Bischof and Kraus, 2021), with parameters α, $\mathbb{E}[\rho] = \beta$, and $\mathcal{T}$ set to 0.95, 0.99, 1, respectively. When using ReLoBraLo, weights are iteratively updated according to the single sub-loss history to balance contributions. Finally, Stochastic Gradient Descent (SGD), or other optimization algorithms, are performed through backpropagation to minimize the PDE residual and train the NN.

To evaluate the applicability of PINN in solving the problems defined in Section 3, different neural architectures with different activation functions have been implemented. More specifically, we initially employed a PINN architecture based on a simple feed-forward network, i.e., a multilayer perceptron (MLP). Then, the so-called Modified Fourier Network (MFN) (MFN Lib. and NVIDIA Corporation) has been used. It is a modified version of network exploiting Fourier features (Tancik et al., 2020). Such architecture has been devised to overcome any *spectral bias* (i.e., the difficulty in learning high-frequency functions) that typically afflicts MLP (Rahaman et al., 2019). A feature encoding layer is used to project the input feature into a higher dimensional space using high-frequency functions. In the Modulus library (NVIDIA Modulus lib.) used in our solution, such encoding is learned during training and in the MFN two additional transformation layers are used to further transform the Fourier features into another feature space to improve convergence in training and final accuracies. Further details on the MLP and MFN architectures are reported in Appendix A.

Regarding activation functions, a number of solutions have been assessed, namely: the Hyperbolic Tangent (Tanh), the Swish function (or SILU, Sigmoid Linear Unit) (Dung et al., 2023), the Gaussian Error Linear Unit (GELU) (Hendrycks and Gimpel, 2016) and considering non-adaptive and adaptive activations (Jagtap et al., 2020). Optimizations have been carried out using ADAM (adaptive moment estimation), with an initial learning rate (lr_{start}) of 0.001 and an exponential decay with $\gamma = 0.999947$ (Wang et al., 2023a), such that

$$lr_{step} = lr_{start} \bullet \gamma^{step} \tag{21}$$

Since our setups consider a temporal evolution with $t \in [0,3]$, solving the problems on the whole-time window at once would lead to poor results. This has been assessed in some preliminary tests where a single time window has been used, confirming the findings presented in the literature (Wight and Zhao, 2020; Krishnapriyan et al., 2021). See Appendix B for further discussion on single vs multiple time windows and an experiment. Such disappointing results are mainly due to an insufficient number of temporal samples. To avoid the use of a fixed sampling, that can be used to ensure a sufficient time coverage and would lead to inaccuracies in not considered locations, we defined smaller time windows in which space-time points are randomly sampled with a sufficient density on both time and space. Moreover, in this way each time window works on a normalized time span in [0,1] that helps to better deal with numerical stability. Thus, an approach with a set of sequential moving time windows has been used to reduce the problem complexity, splitting the time range in multiple T windows (with $T = 3$ in our cases). For each time window a network with the same architecture and hyperparameters is trained anew, using as initial conditions for the sampling points at $t = 1$ and $t = 2$ the final estimates obtained in the previous time step. The maximum number of training steps for each time window has been set to 100000.

The number of sampling/collocation points per step on which to evaluate the losses has been variated in different cases to assess the relation between the achievable accuracy and the training times: indeed, a higher number of sampling points would lead to better accuracies, since more points are constrained to follow the PDEs and the IC and BC. Conversely, having to compute the loss on a huge number of points would increase the computational times. Details and results on such a trade-off are reported in the next sections.

6. Validation of PINN results against CFD/FVM simulations

In this section, the results of the experimental analysis are reported. In order to assess the velocity and pressure estimate accuracy, results produced by the PINN have been compared with the outputs of the FVM solution described in Section 4. The metrics for the assessment are reported in Section 6.1.

The results on the usage of different architectures are provided in Section 6.2. The outcomes obtained by using different activation functions are reported in Section 6.3, while by using different number of neurons, different number of layers are discussed in Section 6.4. These cases have been carried out using the setup with a single box piece in the channel/pipe with small inlets/outlets, represented in Fig. 2(a). Using the best architecture and activation, results of new trainings with the configurations shown in Fig. 2(b) and (c) are reported in Section 6.5. Assessments on using a different number of points are reported in Section 6.6 for all the piece geometries to show the achievable trade-off between accuracy and training times.

Note that PINN inference times, i.e., the times required to obtain a prediction using a trained model are on average less than 6 min for the considered time span, including also the time required to load the sampling points in memory and save the results. Therefore, in the following, we focused on the training times that undoubtfully dominate the computational times required to get CFD simulations with PINN.

For completeness, in addition to the results presented in the following sections, additional video renderings of the experiment results are reported in a web page accessible at the following link https://www.snap4city.org/drupal/node/1010.

All computations have been carried out exploiting a workstation equipped with an Intel Intel(R) Xeon(R) CPU E5-2680 v2 @ 2.80 GHz, with 24 GB or RAM and an NVIDIA GTX4090 GPU.

6.1. Assessment metrics

To measure the differences between the FVM and PINN results, the Mean Absolute Error at time t ($MAE(t')$) is computed according to (22) taking into account the N points uniformly distributed in channel $\mathbb{C}$ used in the FVM simulation (different for each case and with an order of magnitude of 10^4). They are used in the inference phase of the trained PINN (i.e., for the evaluation PINN and FVM work on the same points). $MAE(t')$ is defined as:

$$MAE(t') = \frac{1}{N}\sum_{c=1}^{N} |OF(c,t') - PINN(c,t')| \tag{22}$$

where: $OF(c,t')$ and $PINN(c,t')$ are the results obtained with FVM OpenFOAM and PINN simulations respectively, evaluated in the points $c \in \mathbb{C}$ in each a-dimensional time $t' \in \{0,T\}$, with $T = 3$. Note that: $OF(c,t')$ and $PINN(c,t')$ can be computed on specific variables: velocity magnitude ($\|\mathbf{u}\|$), horizontal (u_x) and vertical (u_y) components of the velocity, and pressure (p). In this way, it is possible to observe the temporal evolution of the errors at each time instant. To assess the error for the whole timespan, the mean MAE over time is used and defined as:

$$MAE_{avg} = \frac{1}{T}\sum_{t'=0}^{T} MAE(t') \tag{23}$$

On the other hand, to estimate the error at steady state (when the transient is finished) we defined the MAE_{st} as:

$$MAE_{st} = \frac{1}{(T-k)}\sum_{t'=k}^{T} MAE(t') \tag{24}$$

where: k has been set to 2.5, since we observed that for $t' \geq 2.5$ the velocity flow fields stable.

6.2. Impact of different network architectures

Results obtained with the different PINN architectures – the Multi-Layer Perceptron (MLP-PINN), and the Modified Fourier Network (MFN-PINN) – are hereafter discussed. For both MLP-PINN, MFN-PINN 6 layers with 512 neurons each have been used. The Tanh activation function has been used, and 4850 points have been adopted in each time window during training. Three consecutive time windows are used.

In Fig. 4, the steady state (after the transient) results of velocity magnitude are shown, while in Fig. 5, error maps for the MLP-PINN and MFN-PINN with respect to OpenFOAM are reported. To appreciate the transient evolution videos of the renderings are accessible on https://www.snap4city.org/1010.

In Fig. 6, the time trend of the $MAE(t')$ on the transient phase computed between the OpenFOAM solution and each of the PINN architectures is provided for both velocity magnitude and pressure. In Table 2, the MAE_{avg} and MAE_{st} for the velocity magnitude, u_x and u_y velocity component, and pressure are reported together with the training times. The MFN-PINN architecture outperforms the MLP-PINN, that shows a diverging error as time increases. Differently, the MFN-PINN quickly converged to low MAE (around 0.02) in particular for the second and last time windows (i.e., $t' > 1$). This result demonstrates the efficacy of the feature transformation applied by the MFN that can better learn the N-S solution. In the above comparison of MLP and MFN architectures, the same number of points has been used. On the other hand, a relevant increment of points for the MLP does not allow to reach similar results as those obtained by using MFN. Therefore, even if the training times for the MFN are longer with respect to those required by the MLP, the differences in performance, the MFN has been adopted for all the subsequent cases.

6.3. Impact of different activation functions

In this section, the results obtained by using different activation functions for the proposed PINN solution are discussed. Solutions have been carried out by using the MFN-PINN architecture with Tanh activations. Additional cases have been conducted by using the SILU and GELU functions. All activation functions have been evaluated also in the *adaptive activation* mode. Errors have been computed comparing our MFN-PINN-based results with respect to the OpenFOAM solution. As shown with previous cases, in Table 3, MAE_{avg}, MAE_{st}, and training times are reported, and in Fig. 7 $MAE(t')$ for $t' \in [0,3]$ velocity and pression are shown. Results demonstrate that there are no strong differences among the activation functions, with the Adaptive GELU obtaining the best average scores for velocity estimates. SILU achieves the best results for the pressure p. Tanh seems to obtain on average the worst results: indeed, Tanh has upper and lower bounds, while GELU and SILU are unbounded, and this can mitigate vanishing gradient problems and help to learn representations with large dynamics. Note that we did not include in this test the ReLU (Rectified Linear Unit) activation. Even if ReLU does not provide an upper bound, like SILU and GELU, it is a non-smooth piecewise linear function that is not differentiable at zero and can lead to inaccurate or suboptimal solutions (Dung et al., 2023). Finally, considering the training times, all the adaptive activations required longer times due to the additional parameters to be learnt. According to these results, the best activation seems to be the adaptive versions of the SILU and GELU, and the adaptive GELU has provided a faster training time. Therefore, in the following, the GELU Adaptive activation function has been used.

6.4. Impact of different layer width and network depth

In order to identify the better ranked architecture an analysis of the impact of the number of neurons of each network layer has been performed. Using the MFN-PINN and the GELU adaptive activation, different layers' widths with 64, 128, 256, and 512 neurons have been assessed. Results are shown in Fig. 8 and Table 4, where transient errors for the velocity magnitude and pressure, and average errors with the training times are reported, respectively.

As a general consideration, the increment of the number of neurons generally leads to lower errors. Configurations with 64 or 128 appeared to be unable to learn the N-S equations, since they resulted in high er-

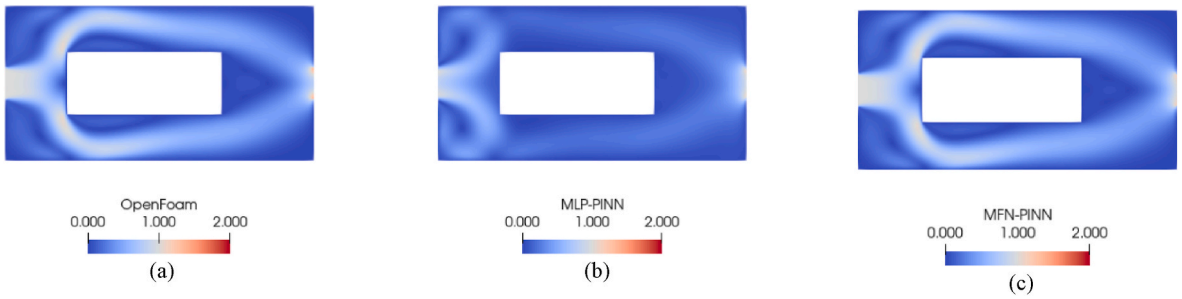


Fig. 4. Steady-state renderings of the velocity magnitude estimates $\|\mathbf{u}\|$ obtained with (a) OpenFOAM, (b) MLP-PINN, and (c) MFN-PINN at time 2.5 for case study of Section 6.2 on neural architecture evaluation. While OpenFOAM and MFN-PINN renderings are practically identical, the MLP-PINN is clearly unable to estimate the correct flows. Please observe Fig. 5 for the related error maps. (Best viewed in color). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)



Fig. 5. Absolute error map for the steady-state results at time 2.5 of the velocity magnitude $\|\mathbf{u}\|$ for cases of Section 6.2 where different neural architectures are evaluated. In (a) the error map of the estimates obtained with the MLP-PINN; in (b) error map obtained with the MFN-PINN. Please note that the color-scale ranges of the error maps are different from the scale of the flows of Fig. 4. When using the MLP-PINN, high errors with respect to the OpenFOAM estimates are observed, while the MFN-PINN is able to obtain very small differences. (Best viewed in color). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

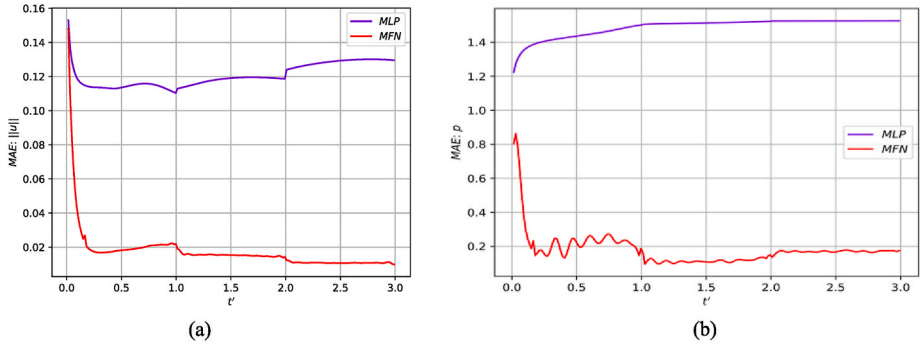


Fig. 6. Mean absolute error, $MAE(t')$, plots for MLP-PINN (in purple) and MFN-PINN (in red) of: (a) velocity magnitude $\|\mathbf{u}\|$, and (b) pressure p. $MAE(t')$ is computed according to eq. (22) exploiting the OpenFOAM estimates. MFN-PINN clearly outperform the MLP-PINN that archived higher errors with and evident growth as time proceeds. (Best viewed in color). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

Table 2
MAE_{avg} and MAE_{st} considering the velocity magnitude ($\|\mathbf{u}\|$), horizontal (u_x) and vertical (u_y) components of the velocity, and pressure (p). The last column reports the training times for the different PINN architectures. Best values (i.e., lower errors/lower times) are highlighted in bold. Even if MLP-PINN, being a simpler architecture, can be trained in shorter times it achieved errors an order of magnitude larger compared to the MFN-PINN that is therefore to be preferred.

Architecture	MAE_{avg}				MAE_{st}				Training Times (hh:mm)
	$\|\mathbf{u}\|$	u_x	u_y	p	$\|\mathbf{u}\|$	u_x	u_y	p	
MLP-PINN	0.121	0.111	0.053	1.489	0.130	0.119	0.054	1.526	**03:32**
MFN-PINN	**0.017**	**0.017**	**0.009**	**0.179**	**0.011**	**0.010**	**0.005**	**0.171**	04:55

rors. Conversely, layers with 256 and 512 neurons obtained very similar errors. In particular, when the system reaches the steady state ($t' > 2.5$). However, having wider layers impacts negatively on the execution time required for the network training passing from 04h:25m (for 256 neurons) to 05h:49m (for 512). For these reasons, in the following reported cases, a layer width of 256 neurons has been used, since it can achieve low errors with reduced training times.

A similar test has been conducted to assess the impact of different network depths by evaluating results using networks composed by 3, 6, and 9 hidden layers. Fig. 9 and Table 5 show the transient errors for both

Table 3
MAE_{avg} and MAE_{st} on the estimates obtained with the MNF-PINN using different activation functions. Errors are reported for velocity magnitude ($\|\mathbf{u}\|$), horizontal (u_x) and vertical (u_y) components of the velocity, and pressure (p). In the last column the training times required for each case are shown. Best values are highlighted in bold. Note that, the results for Tanh are the same as those reported in Table 2 for the MFN-PINN. SILU and GELU (both the adaptive and non-adaptive versions) have obtained very close results. However, the GELU-Adaptive appears to have achieved the best trade-off between high accuracy (low errors) and faster training times, thus it has been selected for the subsequent tests.

Activations	MAE_{avg}				MAE_{st}				Training Times (hh:mm)
	$\|\mathbf{u}\|$	u_x	u_y	p	$\|\mathbf{u}\|$	u_x	u_y	p	
Tanh	0.017	0.017	0.009	0.179	0.011	0.010	0.005	0.171	**04:55**
Tanh Adaptive	0.018	0.017	0.011	0.158	0.010	0.010	0.005	0.119	05:32
SILU	0.014	0.013	**0.007**	**0.107**	0.009	0.008	0.005	**0.093**	05:47
SILU Adaptive	**0.013**	0.013	**0.007**	0.126	**0.008**	**0.007**	**0.004**	0.138	06:10
GELU	**0.013**	0.013	**0.007**	0.166	0.009	0.008	0.005	0.174	05:26
GELU Adaptive	**0.013**	**0.012**	**0.007**	0.145	**0.008**	0.008	**0.004**	0.138	05:49

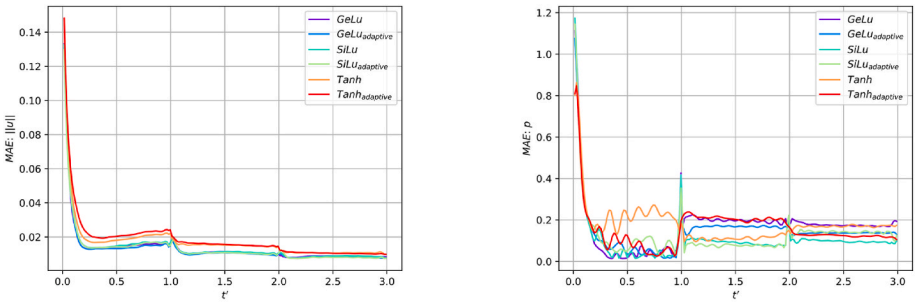


Fig. 7. Plots of $MAE(t')$, computed according to eq. (22), considering in (a) the velocity magnitude $\|\mathbf{u}\|$ and in (b) the pressure p, for the whole-time span ($t \in [0, 3]$) computed with respect to the OpenFOAM solution for the different activation functions. While both versions of Tanh (red and orange plots) show higher $\|\mathbf{u}\|$ errors, SILU and GELU share very close results (see Table 3 to observe more detailed error results). (Best viewed in color). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

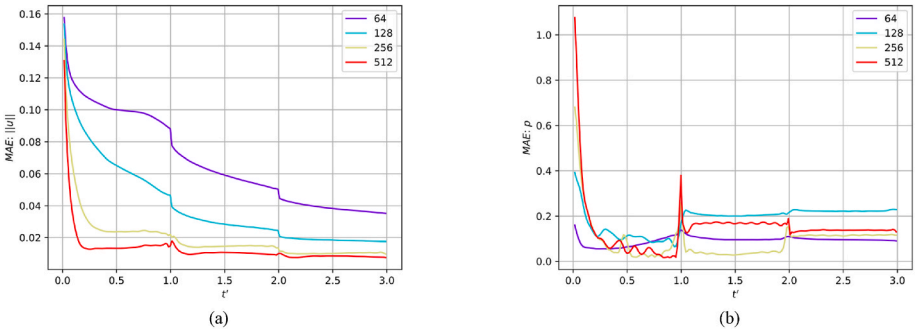


Fig. 8. Plots of $MAE(t')$, computed according to eq. (22), of the velocity magnitude $\|\mathbf{u}\|$ (a) and pressure p (b) computed w.r.t. the OpenFOAM solution for the different layer widths, i.e., 64 (purple), 128 (cyan), 256 (yellow), and 512 (red) neurons. Plots are related to the results presented in Table 4. Except for the first-time window ($t' \in [0, 1]$) the network using layers of 256 neurons achieves very close $\|\mathbf{u}\|$ errors if compared to those of the network with 512 neurons per layer. Due to this small difference and to its faster training times (see Table 4), the solution with 256 has been preferred. (Best viewed in color). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

velocity magnitude and pressure, and the average errors together with their respective training times for the three network depth configurations. The architecture using 3 hidden layers requires an inferior training time and obtains the worst performances, in particular in the first two time windows. Solutions using 6 and 9 hidden layers achieve comparable results, with the latter obtaining slightly lower errors in the transient phase. However, solution with 9 layers increases the training time to 05h:18m with respect to the 04h:25m required by using 6 hidden layers. Therefore, in the following tests the architecture with 6 hidden layers has been used since it provides better compromise of errors and training time.

6.5. Results on different piece geometries

In this section, results on the circular cylinder (see Fig. 2(b)) and the three-boxes (see Fig. 2(c)) piece geometries are presented. We used the MFN-PINN architecture, with 6 layers of 256 neurons each, and GELU adaptive activation function. The networks have been trained from scratch for each experiment. Fig. 10 shows the rendering of the steady state velocity magnitude obtained with OpenFOAM and PINN and the respective error maps. In Table 6, MAE_{avg} and MAE_{st} are reported. Finally, in Fig. 11, the transient error plots of the velocity magnitude and pressure are presented. For comparison purposes, we reported also the results obtained with the single-box piece used in the previous sections. Results are favorable for all the different piece geometries, meaning that

Table 4
MAE_{avg} and MAE_{st} considering estimates for the velocity magnitude ($\|\mathbf{u}\|$), horizontal (u_x) and vertical (u_y) components of the velocity, and pressure (p) obtained with the MFN-PINN, using the GELU-Adaptive activations for the different layer widths (from 64 to 512 neurons for each hidden layer). In the last column training times are reported. In bold, the best values are reported. Note that, the results for 512 are the same as those reported in Table 3 for the GELU Adaptive activation. As expected, the greater the layer width is, the lower are the errors (except for the pressure, as explained in the text) and the longer the training times. Considering the accuracy-time trade-off, we selected the architecture with 256 neurons per layer, achieving low errors (very close to those obtained using 512 neurons) with a substantial saving in the required training times (01h:24m less than the time used by the 512-test case).

Layer width	MAE_{avg}				MAE_{st}				Training Times (hh:mm)
	$\|\mathbf{u}\|$	u_x	u_y	p	$\|\mathbf{u}\|$	u_x	u_y	p	
64	0.068	0.064	0.032	0.092	0.037	0.034	0.018	**0.094**	**03:24**
128	0.040	0.038	0.020	0.187	0.018	0.017	0.009	0.223	03:38
256	0.019	0.019	0.010	**0.091**	0.010	0.010	0.005	0.116	04:25
512	**0.013**	**0.012**	**0.007**	0.145	**0.008**	**0.008**	**0.004**	0.138	05:49

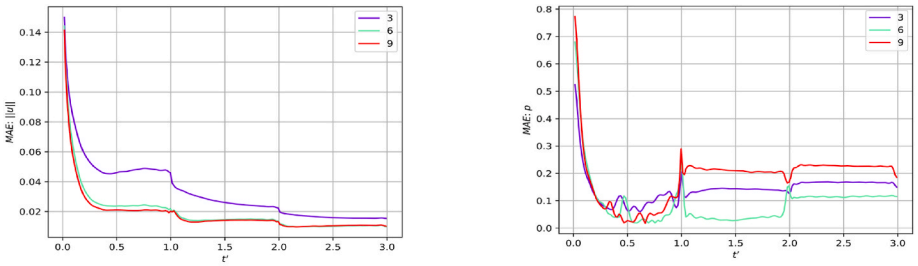


Fig. 9. Plots of $MAE(t')$ of both velocity magnitude $\|\mathbf{u}\|$ (a) and pressure p (b) computed w.r.t. the OpenFOAM solution for the different network depths, i.e., 3 (purple), 6 (cyan), and 9 (red) hidden layers. Using 6 or 9 layers produced very similar results with slightly higher errors for the 6-layer case for $t \in [0,1]$. However, as reported in Table 5, a sensible reduction of training times can be achieved using 6 hidden layers. (Best viewed in color). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

Table 5
MAE_{avg} and MAE_{st} considering both velocity magnitude ($\|\mathbf{u}\|$), horizontal (u_x) and vertical (u_y) components of the velocity, and pressure (p) for the different network depths (3, 6, and 9 hidden layers). In the last column training times are reported. Best values are reported in bold. Note that, the results using 6 layers are the same as those reported in Table 4 for the case with 256 neurons. Using 3 layers resulted in higher errors. Conversely, few differences are appreciable using 6 or 9 layers, and 9 layers require longer training. Thus, the architecture with 6 hidden layers has been selected.

Hidden layer number	MAE_{avg}				MAE_{st}				Training Times (hh:mm)
	$\|\mathbf{u}\|$	u_x	u_y	p	$\|\mathbf{u}\|$	u_x	u_y	p	
3	0.033	0.031	0.017	0.144	0.016	0.015	0.007	0.166	**03:22**
6	0.019	0.019	**0.010**	**0.091**	**0.010**	**0.010**	**0.005**	**0.116**	04:25
9	**0.018**	**0.017**	**0.010**	0.185	**0.010**	**0.010**	**0.005**	0.234	05:18

the MFN-PINN solution has achieved low errors in particular for the steady state. Among the geometries, the case with three-boxes shows the higher velocity errors, followed by the case with single box. This could be due to the sharp borders of those geometries, thus leading to greater difficulties in learning the air flow dynamic, when impacting on the pieces. Indeed, in Fig. 10(c) and (i) the more evident errors are accumulated near the front corners of the pieces. On the other hand, the smoother shape of the cylinder leads to better estimates of the flow velocity. Pressure seems to obtain less accurate predictions, however, analyzing the OpenFOAM results, pressure varies between $[-8.816, 1.390]$, a wider range with respect to the velocity covering the range $[0.000, 1.996]$. Therefore, such errors are acceptable, taking also into account that no specific constraints have been defined for the pressure in the PINN loss.

6.6. Impact of different number of points on accuracy and time

In this section, the trade-off between accuracy and the number of points used is assessed, also considering the impact on the training times. Table 7 shows the different number of points used as *High*, *Mid*, and *Low* numbers/cases. The *Mid* number of points has been adopted for the results of previous sections. In Table 8, MAE_{avg} and MAE_{st} are reported together with the measured training times, while in Fig. 12 the error evolution for all the time instants is presented, as to both velocity and pressure. It can be observed that, as in the previous cases, the circular cylinder appears to be the easiest to solve, achieving the lowest velocity errors. Low errors are anyhow obtained also in the other two geometries, and for all cases velocity errors are reduced, as the number of points increases. Pressure results show fewer regular trends, with lower values not always obtained using a higher number of collocation points. This behavior is due to the fact that we did not impose any pressure constraints on IC and BC.

When using a smaller number of points, training times are reduced, passing from an average of 07h:24m for the *High* case, to 04h:24m for the *Mid* case, and to 02h:47m for the *Low* case. However, MAE_{avg} and MAE_{st} provided relatively small differences between *Mid* and *High* cases. These results suggested that a careful selection of the number of points produces a significant reduction in training times without relevant drops in performance, and for this reason in the following a novel training strategy exploiting such a consideration is proposed (see Section 7.2).

7. Training strategies for faster learning with PINN

As reported in the previous section, using adequate architecture and

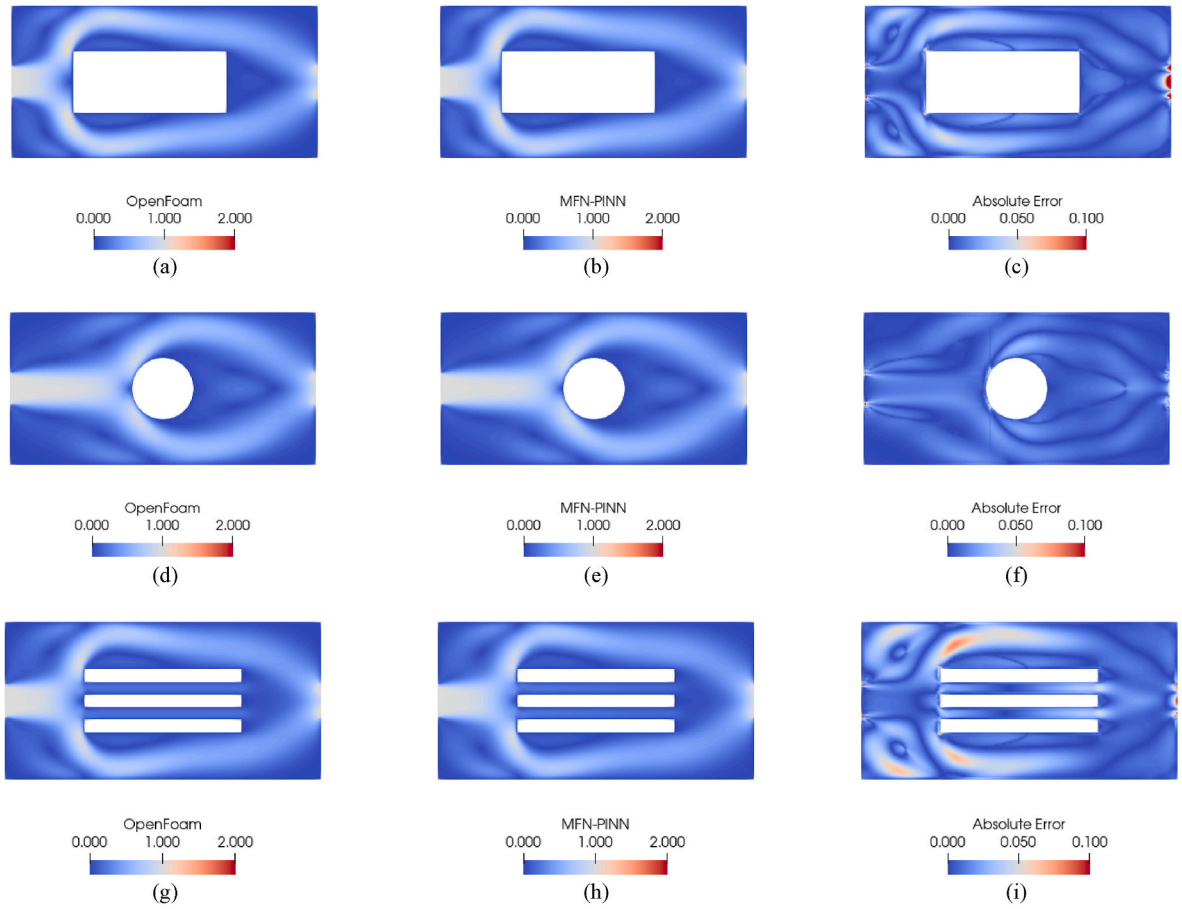


Fig. 10. Rendering of the velocity magnitude $\|\mathbf{u}\|$ results regarding the tests using different pieces inside the channel, as described in Section 6.5. PINN results are obtained using the MFN-PINN with GELU adaptive activation, 6 hidden layers, and 256 neurons for each layer. In the first row the renderings of the single-box case are reported; in the second row the results using the circular cylinder; in the third row those considering the three-boxes. Column wise, the first column (a, d, g) reports the OpenFOAM results, while the second column (b, e, h) shows the MFN-PINN estimates. The PINN estimated flows are very close to those obtained with the OpenFOAM FVM for all the considered piece geometries. To better highlight any differences, in the third column (c, f, i) the absolute error maps, i.e., the pointwise absolute difference of the PINN and OpenFOAM $\|\mathbf{u}\|$ estimates, are reported. Please note that in the error maps a smaller range of values ([0.0, 0.1]) is used in the colormaps to better show the slight differences. Observing the renderings, most of the errors seems to accumulate near the piece border, with the circular cylinder exhibiting the lowest errors due to its more aerodynamic shape compared with the geometries of the single box and three boxes. (Best viewed in color). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

Table 6
MAE_{avg} and MAE_{st} for the test cases using different piece geometries (Section 6.5). Errors are related to both velocity magnitude ($\|\mathbf{u}\|$), horizontal (u_x) and vertical (u_y) components of the velocity, and pressure (p). As illustrated also in Figs. 10 and 11, low errors are achieved for all the geometries with the circular cylinder obtaining the lowest errors.

Piece	MAE_{avg}				MAE_{st}			
	$\|\mathbf{u}\|$	u_x	u_y	p	$\|\mathbf{u}\|$	u_x	u_y	p
Single box	0.019	0.019	0.010	0.091	0.010	0.010	0.005	0.116
Circular cylinder	0.011	0.010	0.006	0.065	0.007	0.006	0.004	0.040
Three boxes	0.024	0.023	0.014	0.048	0.013	0.012	0.006	0.021

activation functions, the proposed PINN results on the three test cases are very satisfactory, showing low differences with respect to the results obtained by OpenFOAM as CFD simulations, both in the transient phase and at steady state. One of the main limitations in the usage of PINN in real scenarios seems to be the training times (see Tables 4, 5 and 8). Therefore, in this section, alternative training techniques with the objective of reducing the time required to train neural networks are proposed.

- **Case A) Fine-tuning**: in Section 7.1 the circular cylinder and the three-boxes geometries have been addressed by fine-tuning of the model trained on the single box to assess the possibility of using a pre-trained network to solve different problems and reduce training times. Moreover, results on fine-tuning between consecutive time windows for the single box geometry are reported.
- **Case B) Multi-resolution training**: in Section 7.2 we proposed a training solution named multi-resolution in which the number of points used varies in the time windows. Under the hypothesis of being more interested in simulating the steady state of the solutions, we trained the first two time-windows with fewer points, rising their numbers in the final window. Note that, this is not possible in CFD

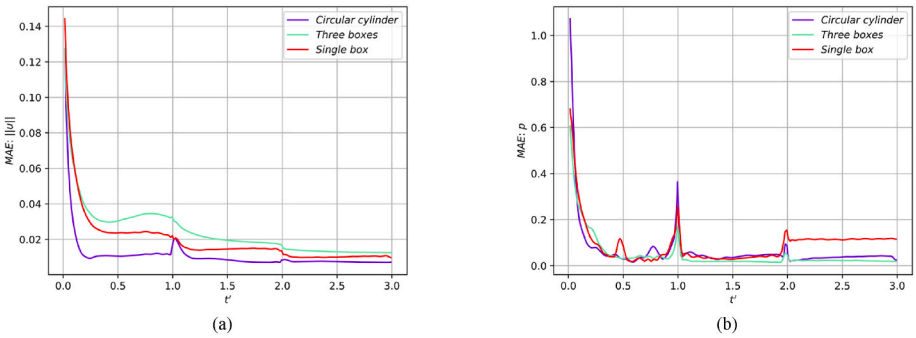


Fig. 11. Plots of $MAE(t')$ for the test cases considering different piece geometries: circular cylinder (purple), three-boxes (cyan) and single box (red). In (a) plots of velocity magnitude $\|\mathbf{u}\|$ errors; in (b) those considering the pressure p. Plots are related to renderings shown in Fig. 10, and errors reported in Table 6. Observing the errors during the time span ($t' \in [0, 3]$), the circular cylinder is confirmed to be the easiest test case with the lowest errors. The single box and the three-boxes, even though showing higher errors due to their more complex geometries, still obtain very good accuracies. (Best viewed in color). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

Table 7
Different numbers of collocation points used in the tests are described in Section 6.6. In the table, the number of points used for each specific area of the considered setup are reported (inlet, outlet, channel walls, piece walls, and in the interior part of the channel where the fluid flow moves). In the last row, the total number of points is reported.

Area	High	Mid	Low
Inlet	500	250	100
Outlet	500	250	100
Channel walls	2500	1250	500
Piece walls	1250	600	250
Channel interior	5000	2500	1000
Total	9750	4850	1950

solvers, where the mesh grid must be kept fixed for the whole simulation.

- **Case C) Parametrized training:** in Section 7.3, we proposed to use a parametrized representation of the piece geometry. In this way it is possible to train a PINN model to solve a larger number of setups with pieces of different dimensions.

7.1. Case A) fine-tuning results

In this case, the results on the circular cylinder and three-boxes test cases have been obtained by fine-tuning the network model trained for computing the solution of the single box case. The analysis estimated the errors at different numbers of epochs, assessing any possible reduction of the training times. Different learning rates have been evaluated, the same used in the previous version ($lr = 10^{-3}$) and one with an order of magnitude larger ($lr = 10^{-2}$). To achieve this strategy, the sequential solver of Modulus (SeqSolver lib.) has been modified to introduce the possibility to re-load weights from a previously trained model. This experiment is particularly useful to assess the applicability of PINN in real scenario, where a single well-trained model could be deployed in different domains with lower effort, thus reducing time and simulation costs.

MAE_{avg} and MAE_{st} errors are reported in Table 9, and transient errors in Fig. 13. We reported errors at different number of epochs to highlight possible gains in training times compared with the measured errors. For the circular cylinder at $lr = 10^{-3}$, starting from 40000 epochs the trained model already achieves velocity errors very similar to the model trained from scratch (discussed in Section 6.5 and Table 6), and after reaching 60000 epochs, errors are equals or even lower using fine-tuning. Note that by increasing the learning rate a slight precision improvement of velocity estimation is obtained with more than 60000 epochs. A similar behavior can be observed for the three-boxes geometry, with comparable or lower errors starting from 40000 epochs. Pressure shows a more erratic behavior as above discussed.

With the approach proposed, training times have been significantly reduced, passing from 04h:24m for training the model from scratch, to 02h:01m when fine-tuning for 40000 epochs, to 03h:00m for 60000 epochs. Note that, a small overhead in the measured times has been obtained, since to get all the results at different epochs reported we incrementally augmented the number of epochs: for example, the model trained till 40000 epochs has been saved and then reloaded to be used as a starting point to perform additional training steps up to 60000 epochs causing an overhead due to saving and loading. When training in a single

Table 8
MAE_{avg} and MAE_{st} for the different geometries and different numbers of collocation points (see Section 6.6). Errors consider both velocity magnitude ($\|\mathbf{u}\|$), horizontal (u_x) and vertical (u_y) components of the velocity, and pressure (p). In the last column training times (in hours and minutes) are reported. Best values for each different geometry are reported in bold. Increasing the number of sampling points helps to obtain more accurate estimates at the expense of higher training times, since the loss function is evaluated on a greater number of points. These results highlight the relevance of choosing an adequate number of points, particularly for the more complex geometries like the single box and the three-boxes cases.

Geometry	Collocation points	MAE_{avg}				MAE_{st}				Training times (hh:mm)
		$\|\mathbf{u}\|$	u_x	u_y	p	$\|\mathbf{u}\|$	u_x	u_y	p	
Single box	Low	0.024	0.023	0.013	0.071	0.013	0.013	0.006	0.048	**02:47**
	Mid	0.019	0.019	0.010	0.091	0.010	0.010	**0.005**	0.116	04:25
	High	**0.017**	**0.016**	**0.009**	0.089	**0.009**	**0.008**	**0.005**	**0.068**	07:14
Circular cylinder	Low	0.012	0.012	0.007	0.128	**0.007**	**0.006**	**0.004**	0.166	**02:47**
	Mid	0.011	**0.010**	**0.006**	**0.065**	**0.007**	**0.006**	**0.004**	**0.040**	04:24
	High	**0.010**	**0.010**	**0.006**	0.219	**0.007**	**0.006**	**0.004**	0.226	07:11
Three boxes	Low	0.028	0.026	0.016	0.059	0.015	0.014	0.008	0.036	**02:48**
	Mid	0.024	0.023	0.014	0.048	0.013	0.012	0.006	**0.021**	04:24
	High	**0.021**	**0.019**	**0.012**	**0.112**	**0.010**	**0.009**	**0.005**	0.126	07:47

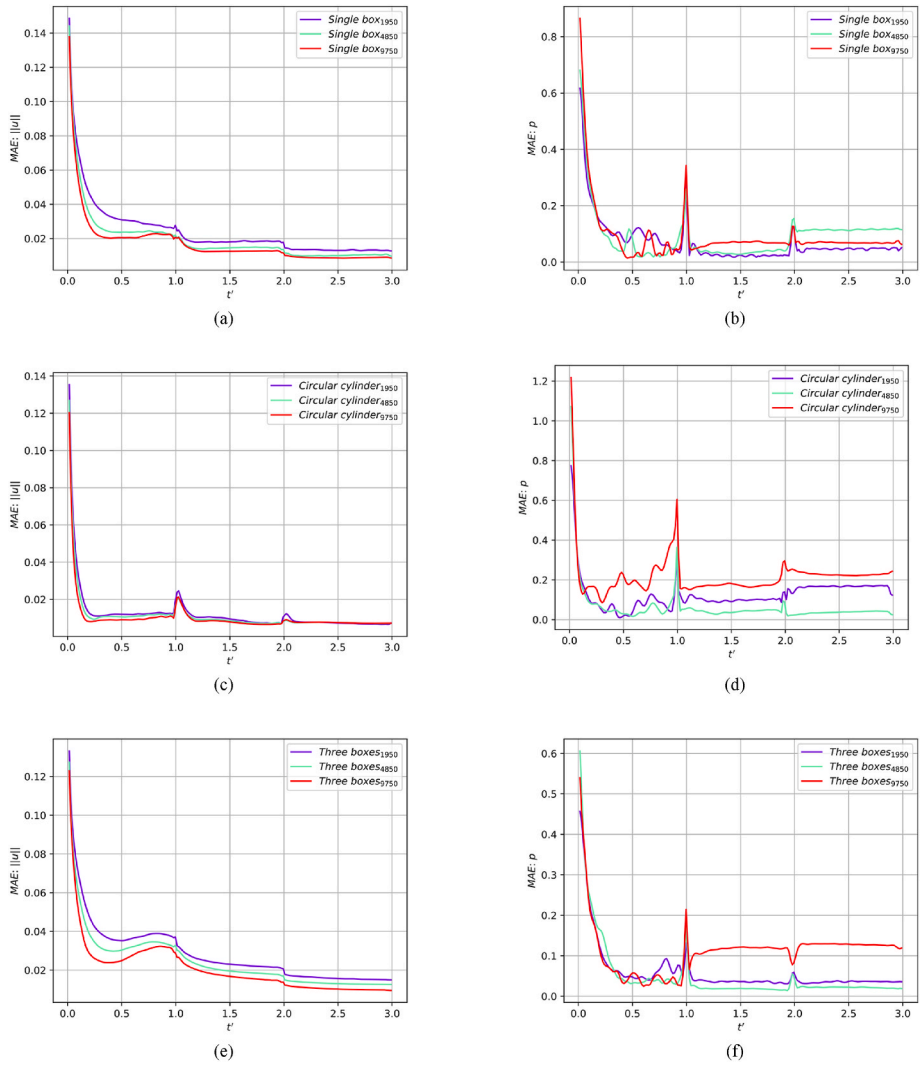


Fig. 12. Plots of $MAE(t')$ for results using different number of collocation points described in Section 6.6, and Table 8. In (a) and (b) plots of velocity magnitude $\|\mathbf{u}\|$ and pressure p error, respectively, for the single box geometry. Similarly, plots for the circular cylinder are shown in (c) and (d), while for the three-boxes are depicted in (e) and (f). All the plots have confirmed that the PINN can achieve accurate estimates, in line to those obtained with OpenFOAM FVM. It can be noted that the circular cylinder is less affected by the different number of points. Conversely, slight improvements are achieved in the single box and three-boxes test case, when using a higher number of points. (Best viewed in color). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

run up to a given number of epochs, by skipping intermediate savings and loadings, the total training times are lower.

A second application of fine-tuning is to exploit the model trained for a time-window and fine-tune it in the successive time window. Let us now use the single-box geometry. Let $\mathscr{M}^0$ be the trained model for the first time-window (obtained with the solution described in Section 6.5). Then, model $\mathscr{M}^1$ of the second time window has been obtained by fine-tuning of $\mathscr{M}^0$. Similarly, the third time window model $\mathscr{M}^2$, has been obtained by fine-tuning $\mathscr{M}^1$.

In Table 10 MAE_{avg}, MAE_{st}, and training times (considering the time required to train the 2nd and the 3rd windows) are reported. In Fig. 14, transient plots are shown. Results Identical to those obtained by training each time window from scratch have been achieved by fine-tuning for 100000 epochs (except for the pressure that improved in the fine-tuning case), and by fine-tuning for 50000 epochs, errors are very close to those obtained in the training from scratch case. Moreover, by using 50000 fine-tuning epochs, training times halve from 03h:17m to 01h:38m, with a speed-up of 2.

In conclusion, fine-tuning is a successful approach to reduce training times and to obtain similar or better results with respect to any training of the network from scratch, both by moving to different geometry setups or between successive time windows.

7.2. Case B) multi-resolution training

In this section, results using the proposed multi-resolution training approach are presented. In this case, the goal has been to identify a solution to accurately simulate the FD behavior in the last time instants (less importance is given to the first steps of the transient phase). This approach can be an interesting feature of proposed PINN which could shorten the time needed to produce the results of classic CFD simulation, in which the mesh grid must remain fixed and require to estimate velocity and pressure for each time instant in order to estimate/propagate the data to the successive one.

In this case, the first two time-windows have been trained, by using a reduced number of points. On this view, two multiresolution cases have been analyzed, namely: (a) using the *Low* number of points (i.e., 1950); (b) reducing the number of points to 1000 (Inlet:75, Outlet: 75, Channel Walls: 250, Piece Walls: 100, Channel Interior: 500). Please note that in both cases, in the third final window the points are those of the *High*

Table 9
MAE_{avg} and MAE_{st} for fine-tuning starting from the model trained on the single box geometry and described in Section 7.1. In the top part the results for circular cylinder, in the bottom part those for three-boxes. In both cases different learning rates (lr) have been considered. Errors include both velocity magnitude ($\|\mathbf{u}\|$), horizontal (u_x) and vertical (u_y) components of the velocity, and pressure (p). Training times, for different number of fine-tuning epochs, are shown in the last column. In bold best values are described, according to the different kind of transfer learning. Low errors are achieved in both cases starting from 40000 epochs with a relevant reduction of training times if compared with those required to train the model from scratch. With additional epochs, using transfer-learning, the solution shows errors lower than those yielded with a training from scratch.

Geometry	Training type	Training epochs	MAE_{avg}				MAE_{st}				Training Times (hh:mm)
			$\|\mathbf{u}\|$	u_x	u_y	p	$\|\mathbf{u}\|$	u_x	u_y	p	
Single Box	Fine-tuning ($lr = 10^{-3}$)	5000	0.067	0.065	0.041	0.188	0.064	0.062	0.039	0.240	**00:13**
to		10000	0.042	0.040	0.025	0.209	0.035	0.033	0.022	0.270	00:29
Circular cylinder		20000	0.023	0.022	0.014	0.229	0.018	0.017	0.011	0.310	00:59
		40000	0.012	0.012	0.007	0.248	0.008	0.007	0.005	0.342	02:01
		60000	0.010	0.010	0.006	0.252	**0.007**	**0.006**	**0.004**	0.350	03:00
		80000	0.010	0.009	0.005	0.248	**0.007**	**0.006**	**0.004**	0.346	04:01
		100000	0.009	0.009	0.005	0.247	**0.007**	**0.006**	**0.004**	0.343	05:01
	Fine-tuning ($lr = 10^{-2}$)	5000	0.070	0.067	0.037	0.205	0.074	0.071	0.039	0.222	**00:13**
		10000	0.045	0.042	0.025	0.313	0.046	0.044	0.027	0.380	00:29
		20000	0.023	0.021	0.014	0.457	0.022	0.021	0.013	0.585	00:59
		40000	0.014	0.013	0.008	0.561	0.012	0.011	0.007	0.722	02:01
		60000	0.009	0.009	0.005	0.568	**0.007**	**0.006**	**0.004**	0.705	03:01
		80000	**0.008**	**0.008**	**0.004**	0.548	**0.007**	**0.006**	**0.004**	0.651	04:01
		100000	**0.008**	**0.008**	**0.004**	0.531	**0.007**	**0.006**	**0.004**	0.609	05:02
	Training from scratch	100000	0.011	0.010	0.006	**0.065**	**0.007**	**0.006**	**0.004**	**0.040**	04:24
Single Box	Fine-tuning ($lr = 10^{-3}$)	5000	0.091	0.083	0.042	0.088	0.087	0.078	0.04	0.103	**00:13**
to		10000	0.069	0.064	0.033	0.096	0.063	0.058	0.026	0.116	00:29
Three boxes		20000	0.046	0.043	0.023	0.135	0.037	0.034	0.017	0.165	00:59
		40000	0.027	0.025	0.015	0.168	0.019	0.018	0.009	0.214	02:01
		60000	0.020	0.018	0.011	0.181	0.012	0.012	0.006	0.233	03:01
		80000	0.017	0.016	0.010	0.186	0.010	0.009	0.005	0.24	04:01
		100000	0.016	0.015	0.010	0.187	0.009	0.008	0.005	0.242	05:02
	Fine-tuning ($lr = 10^{-2}$)	5000	0.089	0.083	0.038	0.117	0.078	0.072	0.033	0.091	**00:13**
		10000	0.080	0.074	0.036	0.084	0.076	0.069	0.034	0.059	00:29
		20000	0.044	0.041	0.022	0.202	0.040	0.038	0.017	0.157	00:59
		40000	0.027	0.025	0.014	0.235	0.021	0.02	0.009	0.182	02:01
		60000	0.016	0.015	0.009	0.244	0.012	0.011	0.005	0.197	03:01
		80000	0.012	**0.011**	0.007	0.236	0.008	0.007	**0.004**	0.192	04:01
		100000	**0.011**	**0.011**	**0.006**	0.231	**0.007**	**0.006**	**0.004**	0.190	05:02
	Training from scratch	100000	0.024	0.023	0.014	**0.048**	0.013	0.012	0.006	**0.021**	04:24

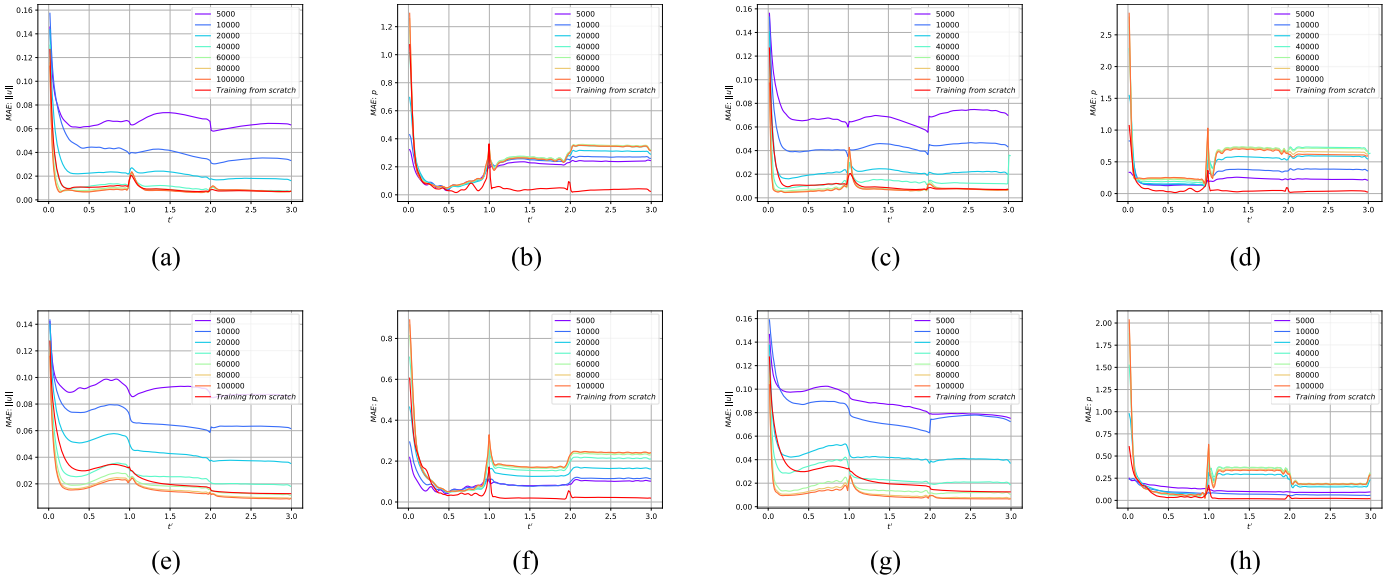


Fig. 13. Plots of $MAE(t')$ for the fine-tuning tests starting from a model trained on the single box geometry (Section 7.1, Table 8). In (a) and (b) velocity magnitude $\|\mathbf{u}\|$ and pressure p errors for the circular cylinder using $lr = 10^{-3}$; in (c) and (d) the same test using $lr = 10^{-2}$. Similarly, in (e) and (f) velocity magnitude $\|\mathbf{u}\|$ and pressure p errors for the three-boxes using $lr = 10^{-3}$; in (g) and (h) using $lr = 10^{-2}$. In all the plots errors related to the estimates obtained with a model trained from scratch are reported in red. By augmenting the number of fine-tuning epochs more accurate results are achieved in all the cases. Moreover, 40000 epochs are sufficient to obtain pretty accurate results, while after 60000 epochs results are practically equals to those obtained by training the model from scratch. (Best viewed in color). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

Table 10
MAE_{avg} and MAE_{st} for the fine-tuning between time windows experiment described in Section 7.1. The table shows errors for both velocity magnitude ($\|\mathbf{u}\|$), horizontal (u_x) and vertical (u_y) components of the velocity, and pressure (p). In the last column the training times for the different number of epochs used to train the 2nd and 3rd time windows are reported. In the last row, the errors obtained by training from scratch are reported for comparative ends. Best values are shown in bold. Note that, at the expense of a slight reduction in accuracy, using 50000 epochs to fine-tune the 2nd and 3rd time windows, training times are halved, thus obtaining a speed-up of 2.

Training epochs	MAE_{avg}				MAE_{st}				Training Times for 2nd and 3rd windows (hh:mm)
	$\|\mathbf{u}\|$	u_x	u_y	p	$\|\mathbf{u}\|$	u_x	u_y	p	
50,000	0.012	0.011	0.007	0.066	0.008	0.007	0.005	0.044	**01:38**
100,000	**0.011**	0.011	**0.006**	**0.055**	**0.007**	**0.006**	**0.004**	0.022	03:17
Training from scratch	**0.011**	**0.010**	**0.006**	0.065	**0.007**	**0.006**	**0.004**	**0.040**	03:17

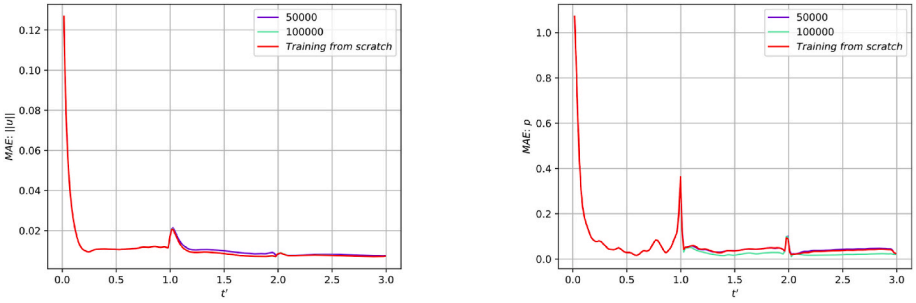


Fig. 14. Plots of $MAE(t')$ for the fine-tuning between successive time windows experiment (see Section 7.1, Table 10). Results are obtained considering the single box geometry. In (a) plot of the velocity magnitude $\|\mathbf{u}\|$ errors, in (b) the errors on pressure p with different plots according to the used number of epochs (violet for 50000, cyan for 100000). In red errors of the estimates from the model trained from scratch. Note that, using 50000 fine tuning epochs accurate results are achieved (with slight difference with respect to. 100000 epochs and the training from scratch). (Best viewed in color). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

level (i.e., 9750).

Table 11 reports the achieved MAE_{avg} and MAE_{st} together with the number of points used in each time windows and the related training times (that for the multi-resolution cases are divided in the times required for the first two windows, and the times employed to train the last window). Transient error plots are shown in Fig. 15. As expected, transient errors are higher with respect to the results obtained by training the model from scratch since less collocation points have been used in the first time-windows. Conversely, steady state results (MAE_{st}) are practically identical to the case with 3 consecutive trainings, demonstrating the validity of the multi-resolution approach for shortening the execution time and obtaining equivalent results. Therefore, a reduction of training time has been achieved passing from the 04h:25m to 03h:50m in case (b).

7.3. Case C) parametrized learning

Since in PINNs the grid of points/mesh do not need to be defined, an additional advantage can be recovered from the possibility of providing the geometry in a parametrized manner. For example, by defining ranges for the element dimensions instead of fixed numbers. This approach opens the possibility to train a single PINN model capable of providing a range of solutions and setups. To assess this capability, the single box case has been adopted defining the box dimensions with the following ranges instead of the fixed values described in Section 3. The horizontal dimension varies in the range $[0.4, 0.6]$ and the vertical dimension in the range $[0.15, 0.25]$. On this basis, a new MFN-PINN model has been trained with the hyperparameters identified, using the adaptive GELU activation, and the *High* number of points. Differently from the previous cases, the maximum training epochs has been increased to 150000 rising the training times to 18h:23m. However, considering the wide range of possible setups that can be solved, such an increase in computational times is acceptable. For example, suppose that in a production line an autoclave is used to cure 10 pieces with a similar shape and different dimensions. According to the regular approach a different PINN should be trained for each different piece requiring 04h:24m. Then the whole set of 10 would require a total training time of about 44 h. On the contrary, using the proposed parametrized approach for training, a speed-up greater than 2 would be achieved.

To evaluate the achieved results, a set of new OpenFOAM simulations have been carried out changing the dimensions of the piece inside the channel, named *Small* and *Big*, with dimensions [0.4,0.15] and [0.6,

Table 11
MAE_{avg} and MAE_{st} for the multi resolution training strategy presented in Section 7.2. Errors are reported for the velocity magnitude ($\|\mathbf{u}\|$), horizontal (u_x) and vertical (u_y) components of the velocity, and pressure (p). In the last column training times are reported. Note that for the multi-resolution cases, training times are divided in the times required for the first two windows, plus the times employed to train the last window. In the last row, errors and times related to a model trained from scratch are shown for comparison. The case (a) and (b) indicate the different number of points used in training the first two windows: 1950 points in (a), 1000 points in (b). Best values are reported in bold. While the multi-resolution approaches obtain higher MAE_{avg} errors that considers the whole-time span, MAE_{st} errors, limited to the steady state, are comparable to those obtained by model trained from scratch.

Experiment	MAE_{avg}				MAE_{st}				Training Times (hh:mm)
	$\|\mathbf{u}\|$	u_x	u_y	p	$\|\mathbf{u}\|$	u_x	u_y	p	
Multi resolution (a)	0.026	0.024	0.014	0.057	0.011	0.011	0.006	0.026	01:57 + 02:14
Multi resolution (b)	0.030	0.029	0.015	0.110	0.012	0.012	0.006	**0.101**	**01:36 + 02:14**
Training from scratch	**0.019**	**0.019**	**0.010**	**0.091**	**0.010**	**0.010**	**0.005**	0.116	04:25

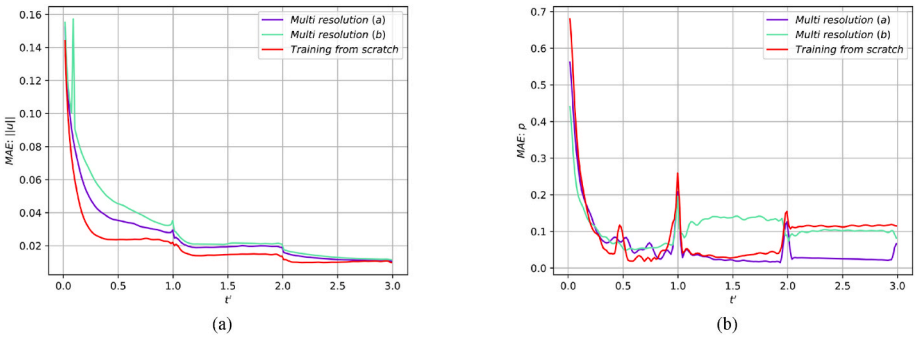


Fig. 15. Plots of $MAE(t')$. In (a) the velocity magnitude $\|\mathbf{u}\|$ errors; in (b) the pressure p errors computed with respect to the OpenFOAM solution for different multi resolution trainings (in purple the (a) case using 1950 points in the first two windows, in cyan the case (b) using 1000 points). In red the reference error plot obtained from the model trained from scratch. Note that, in particular for $\|\mathbf{u}\|$, the estimates of the multi-resolution approaches show higher errors in the first-time windows ($t' \in [0,2]$), while they converge to values comparable to those obtained with the model trained from scratch, achieving practically the same errors, when the system reaches the steady state ($t' \geq 2.5$). (Best viewed in color). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

0.25] respectively, comparing the results obtained with the parametrized approach against the results on the same setup obtained by training the model from scratch (similarly as done in Section 6.5) using the *Mid* number of collocation points and 100000 epochs.

Transient errors are presented in the plots of Fig. 16, and MAE_{avg} and MAE_{st} in Table 12. Thus, training from scratch seems to produce better results (with the exception of pressure estimates that as in the previous case tests show a less regular behavior for the reasons previously discussed). However, there are small differences, in particular when reaching the steady state. Moreover, in the case of the *Small* rectangle differences are smaller, and this can give hints on the fact that the less is the area of the piece inside the channel, minor is its impact on the flow dynamics and smaller are the errors achieved in the transient phase. In conclusion, the parametrized training can be a valid solution to be adopted in scenarios where similar pieces of different dimensions are placed inside the channel.

8. Conclusions

In this study, we explored the application of PINNs to complex fluid dynamic scenarios, focusing on self-supervised setups that exploit solely physical laws without any data. Our work introduced several novel strategies to enhance the efficiency and accuracy of PINNs, particularly when dealing with the Navier-Stokes equations in various geometries inspired by industrial autoclave setups. All the PINN results have been validated against those obtained by using OpenFOAM, a widely used CFD FVM tool. The comparison included assessing errors in velocity magnitude, horizontal and vertical velocity components, and pressure.

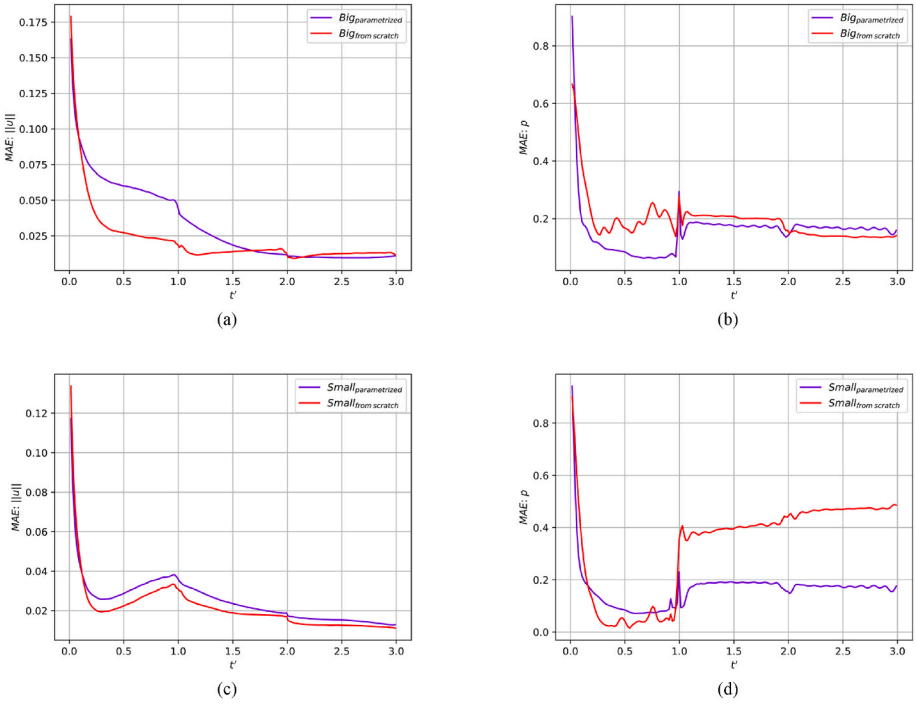


Fig. 16. Plots of the $MAE(t')$ for the parametrized learning tests presented in Section 7.3. In (a) and (b) velocity magnitude $\|\mathbf{u}\|$ and pressure p errors for the *Big* rectangle. In (c) and (d) the same errors for the *Small* rectangle. In all the plots, results of the parametrized learning are reported in violet, while errors obtained by training from scratch using the differently sized rectangles are shown in red. It can be noted that, the parametrized training results are compared with the results obtained by training the network from scratch, in particular when the steady state is reached. (Best viewed in color). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

Table 12
MAE_{avg} and MAE_{st} for the differently sized geometries used in the parametrized learning test case (see Section 7.3 and Fig. 16). For both the *Big* and *Small* rectangles, errors with respect to OpenFOAM are reported considering the parametrized strategy and the training from scratch. Errors consider the velocity magnitude ($\|\mathbf{u}\|$), horizontal (u_x) and vertical (u_y) components of the velocity, and pressure (p). Lower errors are highlighted in bold for each test case. Results indicate that using parametrized learning good estimates are achieved comparable with those yielded by a model trained from scratch, in particular for the steady state (MAE_{st}).

Geometry	Training type	MAE_{avg}				MAE_{st}			
		$\|\mathbf{u}\|$	u_x	u_y	p	$\|\mathbf{u}\|$	u_x	u_y	p
Big rectangle	Parameterized	0.032	0.030	0.015	**0.157**	**0.010**	**0.009**	**0.005**	0.163
	Training from scratch	**0.022**	**0.021**	**0.010**	0.192	0.013	0.012	**0.005**	**0.137**
Small rectangle	Parameterized	0.025	0.023	0.017	**0.165**	0.014	0.013	0.009	**0.172**
	Training from scratch	**0.021**	**0.020**	**0.014**	0.331	**0.012**	**0.011**	**0.007**	0.474

We conducted a large set of experiments to select the optimal neural architecture and hyperparameters. The study evaluated different neural network architectures, such as MLP-PINN and MFN-PINN, and activation functions like Tanh, SILU, and GELU. The MFN-PINN architecture consistently outperformed the MLP-PINN, demonstrating its efficacy in learning the Navier-Stokes solutions. Adaptive GELU activation functions yielded the best average scores for velocity estimates, while SILU excelled in pressure estimates. Note that in future work the adoption of trainable activation function could be considered (Wang et al., 2023b).

Then, additional piece geometries have been considered to evaluate PINN performance widening the test cases. Using the identified architecture (MFN-PINN with adaptive GELU activation) rapid convergences to low MAE both during the transient phase and in the steady state phase have been observed for all the different setups. This further validates PINN performance that show small differences with respect to estimates obtained with CFD methods using the FVM solver. These comparisons established a baseline performance of PINNs in replicating complex fluid dynamics scenarios.

Since one of the primary identified limitations of PINN applicability in real-world scenarios has been the extensive training time required, further test and novel training strategies have been proposed. First, the impact of the number of points used have been assessed, showing that a careful setup can provide a reduction of training times without strong degradation in the performance. Moreover, we evaluated several alternative training strategies. (A) Fine-Tuning: This approach is beneficial when a well-trained model is available. It requires about half the time needed to train a model from scratch, achieving similar results. (B) Multi-Resolution Learning: This strategy is particularly useful for scenarios where steady state results are more critical than transient ones. Simulation errors for the last time window have been comparable to full training from scratch, indicating its effectiveness in reducing training times. (C) Parameterized Learning: this method proves advantageous when dealing with multiple similar geometries. A single trained model can handle a virtually infinite number of setups with comparable accuracies to those obtained from models trained from scratch. Therefore, our findings indicate that PINNs can significantly reduce computational costs while maintaining accuracy levels comparable to traditional CFD methods. This positions PINNs as a viable solution for industrial applications requiring rapid and accurate fluid dynamic simulations.

However, additional studies are necessary to assess the behavior of PINNs in three-dimensional geometries and evaluate other training strategies to further expedite the training process and simulate longer transient times. Additionally, different PDEs should be considered, such as for example heat equations governing the thermal transfer between the fluid and the pieces inside the channel. Finally, the development of PINN-based solutions fully integrated into industrial processes would be required to effectively exploit such novel methods in real-world applications.

In summary, this study demonstrates that PINNs are effective tools for addressing complex fluid dynamics problems governed by the Navier-Stokes equations. They offer an effective alternative to traditional CFD methods, particularly in scenarios requiring rapid simulations, when a range of pieces have to be used. With ongoing advancements and further validation, PINNs have the potential to revolutionize the field of fluid dynamics simulation, paving the way for broader adoption in practical engineering and industrial applications. By embracing these innovative neural network approaches, we can significantly enhance our capability to simulate and predict fluid behavior in various contexts, ultimately leading to more efficient and accurate designs and processes in engineering disciplines. The integration of PINNs in fluid dynamics simulations presents a transformative approach that combines the strengths of machine learning with the foundational principles of physics. This synergy optimizes computational resources and also opens new avenues for real-time and precise simulations, making it a valuable asset for modern engineering challenges.

CRediT authorship contribution statement

Tommaso Botarelli: Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation. **Marco Fanfani:** Writing – review & editing, Writing – original draft, Visualization, Supervision, Software, Investigation, Formal analysis. **Paolo Nesi:** Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Project administration, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Lorenzo Pinelli:** Writing – original draft, Validation, Software, Formal analysis, Data curation.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Paolo Nesi reports financial support was provided by University of Florence.

Acknowledgements

This research has been funded by CAI4DSA actions (Collaborative Explainable neuro-symbolic AI for Decision Support Assistant), of the FAIR national partnership project on foundations of artificial intelligence, PE 1 PNRR (https://fondazione-fair.it/). Snap4City (https://www.snap4city.org/) is an open technology and framework of DISIT Lab (https://www.disit.org).

Appendix A. Multi-Layer Perceptron (MLP) and Modified Fourier Network (MFN) architectures

Hereafter a brief description of the MLP and MFN architectures is provided showing the difference between them. As widely known, a MLP is composed by an input layer, an output layer, and a variable number of hidden layers. Moving from one layer to the next one, input variables are

linearly combined using a weight matrix W and a bias vector **b** and then non-linearity is introduced by means of some activation function σ, as shown in the following equation

$$\mathbf{x}^{k+1} = \sigma\left(\mathrm{W}^k\mathbf{x}^k + \mathbf{b}^k\right) \tag{A1}$$

where $\mathbf{x}^k \in \mathbb{R}^{n+1}$ and $\mathbf{x}^{k+1} \in \mathbb{R}^{m+1}$ are the input vector of the k-th layer and the relative output vector, respectively, $\mathrm{W}^k \in \mathbb{R}^{m+1\times n+1}$ is the weight matrix and $\mathbf{b}^k \in \mathbb{R}^{m+1}$ the bias vector of the k-th layer. A graphical representation of a generic layer of the MLP is shown in Fig. 17. Hidden layers are stacked one after the other to obtain the whole network.

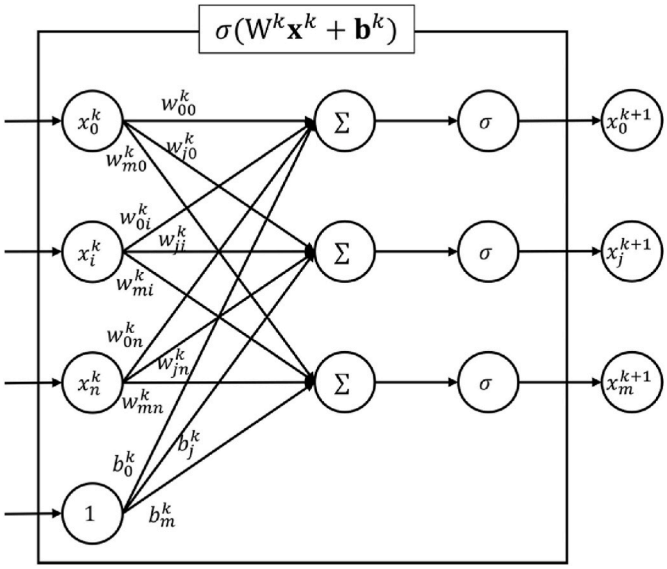


Fig. 17. A generic k layer of a MLP architecture. The input vector $\mathbf{x}^k = \left[x_0^k, \ldots, x_i^k, \ldots, x_n^k\right]^{\mathrm{T}}$ is multiplied with the weights w_{MN} of the matrix W^k (with $N = [0, n]$ and $M = [0, m]$) and the resulting vector is added to the bias vector $\mathbf{b}^k = \left[b_0^k, \ldots, b_j^k, \ldots, b_m^k\right]^{\mathrm{T}}$. A non-linear activation function σ is used and the output vector $\mathbf{x}^{k+1} = \left[x_0^{k+1}, \ldots, x_j^{k+1}, \ldots, x_m^{k+1}\right]^{\mathrm{T}}$ is obtained.

The MFN architecture extends the MLP by introducing an input encoding layer F, and two transformation layers characterized by specific weight matrices W_{T_1} and W_{T_2}, plus bias vectors $\mathbf{b}_{T_1}$ and $\mathbf{b}_{T_2}$, according to the following equation

$$\mathbf{x}^{k+1} = \left(1 - \sigma\left(\mathrm{W}^k\mathbf{x}^k + \mathbf{b}^k\right)\right) \odot \sigma(\mathrm{W}_{T_1}\phi_E + \mathbf{b}_{T_1}) + \sigma\left(\mathrm{W}^k\mathbf{x}^k + \mathbf{b}^k\right) \odot \sigma(\mathrm{W}_{T_2}\phi_E + \mathbf{b}_{T_2}) \tag{A2}$$

where $\odot$ are element-wise multiplications and ϕ_E is obtained from the encoding layer F as

$$\phi_E = \left[\sin\left(2\pi\mathbf{f} \times \mathbf{x}^0\right); \cos\left(2\pi\mathbf{f} \times \mathbf{x}^0\right)\right]^{\mathrm{T}} \tag{A3}$$

In this case, all the W^k and $\mathbf{b}^k$ for each layer plus $\mathbf{f} \in \mathbb{R}^{n_f \times n_0}$ (where n_f is the number of frequency sets and n_0 the dimension of the input vector $\mathbf{x}^0$), $\{\mathrm{W}_{T_1}, \mathbf{b}_{T_1}\}$, and $\{\mathrm{W}_{T_2}, \mathbf{b}_{T_2}\}$ are trainable parameters. In Fig. 18, a schematic representation of the MFN is provided.

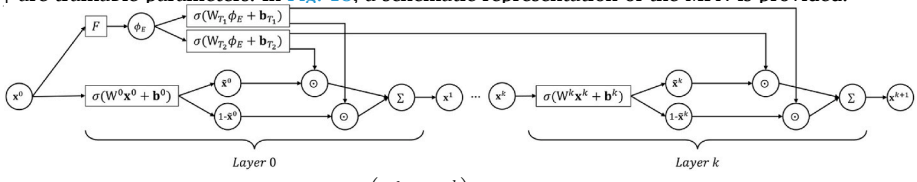


Fig. 18. Schematic representation of the MFN architecture. The block $\sigma\left(\mathrm{W}^k\mathbf{x}^k + \mathbf{b}^k\right)$ is used to represent a MLP layer as shown in Fig. 15. From the input vector $\mathbf{x}^0$ the frequency set ϕ_E is obtained using the encoding layer F. Then, ϕ_E is used as input for the two transformation layers and the results are element-wise multiplied with the outputs of the MLP layers according to equation (A2).

Appendix B. Experimental evidence of the relevance of multiple time windows in PINN training

In this section, we are providing the empirical evidence of the advantages of exploiting multiple time windows in PINN training with respect to the adoption of a single window for the whole time-span, hereafter experimental results are reported. For this experiment, the single box piece has been chosen and an MFN with 6 layers of 256 neurons and GELU adaptive activation has been selected, being the configuration that has obtained the best trade-off between performance and training times (as reported in Sections 6.2, 6.3, and 6.4), excluding the advanced training strategies proposed in Section 7.

For the single time window test, namely *SingleTimeWindow*, 300000 epochs have been used for a fair comparison, since it has been the total number of epochs used in the *MultiTimeWindows* experiment for the same time span of [0,3]. Results on single window have been compared with respect to the training reported in Section 6.4 and Tables 4 and 5 using 6 layers with 256 neurons, named hereafter as *MultiTimeWindows*.

As it can be seen in Table 13 and Fig. 19 reporting the error, the *MultiTimeWindows* produced better results in terms of velocity with respect to the *SingleTimeWindow*. The computational effort in both cases depends on the number of points and the number of epochs, which is identical. The slightly longer times for the *MultiTimeWindows* is due to the overhead for restarting the model when a new time window is started. According to Fig. 19, the *SingleTimeWindow* presents a quite higher error value with respect to the *MultiTimeWindows* approach. This is mainly due to the fact that (i) the physical phenomenon presents quite different behaviour in the first part of the time-span with respect to the rest in which the stationary status arrives (see videos at https://www.snap4city.org/1010 looking at the OpenFoam case of the single box), and (ii) the approach based on *SingleTimeWindow* tries to find a unique model for all the time-span, while having three different trainings as in the *MultiTimeWindows* would produce much more precise estimations.

Moreover, observing Fig. 19, it can be noted that around $t' = 1s$ a relevant reduction of the error is present for both solutions. In this case, two effects have been appreciated. In *SingleTimeWindow* the effect is only associated with specific flow dynamics, which at that time the flow hitting the front of the piece inside the channel reaches its maximum speed and then it stabilizes, making easier the fitting with the stationary model which is dominance in the timespan (see the above-mentioned videos). In the *MultiTimeWindows*, the size of the first-time window has been experimentally aligned with the change of flow behaviour (discussed above), and thus the relevant change is also due to the passage to the new time window (as also occurred at $t' = 2s$, for the third window). Then, when the flow reaches a stable condition in all the channel points, the error is continuously reduced until setting on an almost fixed value after 2.5s (the time we selected to define the steady-state condition, see (24)).

This experimental analysis, better motivate and corroborates the relevance of adopting the *MultiTimeWindows* solutions in PINN training, confirming the literature (Wight and Zhao, 2020; Krishnapriyan et al., 2021).

Table 13
MAE_{avg} and MAE_{st} considering estimates for the velocity magnitude ($\|\mathbf{u}\|$), horizontal (u_x) and vertical (u_y) components of the velocity, obtained for the *MultiTimeWindows*, and *SingleTimeWindow* presented in Appendix B. In the last column, the training times. In bold the best values.

Time Window	MAE_{avg}			MAE_{st}			Training Times (hh:mm)
	$\|\mathbf{u}\|$	u_x	u_y	$\|\mathbf{u}\|$	u_x	u_y	
MultiTimeWindows	**0.019**	**0.019**	**0.010**	**0.010**	**0.010**	**0.005**	04:25
SingleTimeWindow	0.037	0.035	0.017	0.018	0.017	0.010	**04:00**

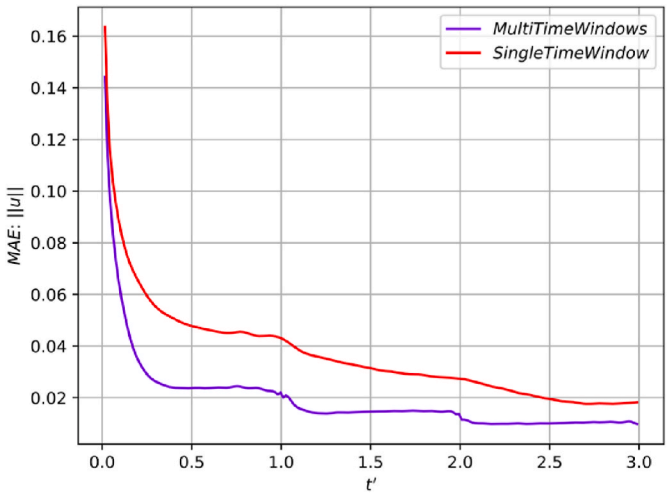


Fig. 19. Plots of $MAE(t')$ of velocity magnitude $\|\mathbf{u}\|$ computed with respect to the OpenFOAM solution for both the *MultiTimeWindows* (violet) and *SingleTimeWindow* (red) configurations discussed in Appendix B. The *MultiTimeWindows* approach outperforms the *SingleTimeWindow* case on estimating $\|\mathbf{u}\|$ for the whole timespan. (Best viewed in color).

Data availability

No data was used for the research described in the article.

References

Barzegar Gerdroodbary, M., 2020. Application of neural network on heat transfer enhancement of magnetohydrodynamic nanofluid. Heat Tran. Res. 49, 197–212. https://doi.org/10.1002/htj.21606.

Baydin, A.G., Pearlmutter, B.A., Radul, A.A., Siskind, J.M., 2018. Automatic differentiation in machine learning: a survey. J. Mach. Learn. Res. 18, 1–43.

Bischof, R., Kraus, M., 2021. Multi-objective loss balancing for physics-informed deep learning. arXiv preprint arXiv:2110.09813.

Cao, Y., Subhashri, A.R., Chandrasekar, A., Radhika, T., Przybyszewski, K., 2024a. Exponential state estimation for delayed competitive neural network via stochastic sampled-data control with Markov jump parameters under actuator failure. J. Artif. Intell. Soft Comput. Res. 14 (4), 373–385.

Cao, Y., Chandrasekar, A., Radhika, T., Vijayakumar, V., 2024b. Input-to-state stability of stochastic Markovian jump genetic regulatory networks. Math. Comput. Simulat. 222, 174–187.

Chuang, P.Y., Barba, L.A., 2022. Experience Report of Physics-Informed Neural Networks in Fluid Simulations: Pitfalls and Frustration arXiv preprint arXiv:2205.14249.

Clyne, T.W., Hull, D., 2019. An Introduction to Composite Materials. Cambridge university press.

De Ryck, T., Jagtap, A.D., Mishra, S., 2024. Error estimates for physics-informed neural networks approximating the Navier–Stokes equations. IMA J. Numer. Anal. 44 (1), 83–119.

Djeddi, S.R., Masoudi, A., Ghadimi, P., 2013. Numerical simulation of flow around diamond-shaped obstacles at low to moderate Reynolds numbers. Am. J. Appl. Math. Stat. 1 (1), 11–20.

Dung, D.V., Song, N.D., Palar, P.S., Zuhal, L.R., 2023. On the Choice of activation functions in physics-informed neural network for solving incompressible fluid flows. In: AIAA SCITECH 2023 Forum, p. 1803.

Eivazi, H., Tahani, M., Schlatter, P., Vinuesa, R., 2022. Physics-informed neural networks for solving Reynolds-averaged Navier–Stokes equations. Phys. Fluids 34 (7).

Faller, William E., Schreck, Scott J., 1997. Unsteady fluid mechanics applications of neural networks. J. Aircraft 34 (1), 48–55.

Fernlund, G., Mobuchon, C., Zobeiry, N., 2018. 2.3 autoclave processing. In: Carl Zweben, P.B. (Ed.), Comprehensive Composite Materials II, pp. 42–62. https://doi.org/10.1016/b978-0-12-803581-8.09899-4.

Ferreira, De Paula, William, Armellini, Fabiano, Santa-Eulalia, Luis Antonio De, 2020. Simulation in industry 4.0: a state-of-the-art review. Comput. Ind. Eng. 149, 106868. https://doi.org/10.1016/j.cie.2020.106868. ISSN 0360-8352.

Fezai, S., Ben-Cheikh, N., Ben-Beya, B., Lili, T., 2020. Numerical study of obstacle geometry effect on the vortex shedding suppression and aerodynamic characteristics. Int. J. Numer. Methods Heat Fluid Flow 30 (2), 469–495.

Flores, F., Garreaud, R., Muñoz, R.C., 2013. CFD simulations of turbulent buoyant atmospheric flows over complex geometry: solver development in OpenFOAM. Computers & Fluids 82, 1–13.

Ghamlouch, T., Roux, S., Bailleul, J.L., Lefèvre, N., Sobotka, V., 2017. Experiments and numerical simulations of flow field and heat transfer coefficients inside an autoclave model. In: AIP Conference Proceedings, vol. 1896. AIP Publishing, 1.

Giaccherini, S., Pinelli, L., Marconcini, M., Pacciani, R., Arnone, A., 2023. Validation of an analytical model for the acoustic Impedance Eduction of Multicavity Resonant liners by a high-fidelity large Eddy simulation approach. ASME Journal of Turbomachinery 145 (8), 081002. https://doi.org/10.1115/1.4056984.

Hendrycks, Dan, Gimpel, Kevin, 2016. Gaussian error linear units (gelus). arXiv Preprint arXiv:1606.08415.

Hirsch, C., 2007. Numerical Computation of Internal and External Flows: the Fundamentals of Computational Fluid Dynamics, second ed. Butterworth-Heinemann.

Hu, B., McDaniel, D., 2023. Applying physics-informed neural networks to solve Navier–Stokes equations for laminar flow around a particle. Math. Comput. Appl. 28 (5), 102.

Jagtap, A.D., Kawaguchi, K., Karniadakis, G.E., 2020. Adaptive activation functions accelerate convergence in deep and physics-informed neural networks. J. Comput. Phys. 404, 109136.

Jin, X., Cai, S., Li, H., Karniadakis, G.E., 2021. NSFnets (Navier-Stokes flow nets): physics-informed neural networks for the incompressible Navier-Stokes equations. J. Comput. Phys. 426, 109951.

Krishnapriyan, A., Gholami, A., Zhe, S., Kirby, R., Mahoney, M.W., 2021. Characterizing possible failure modes in physics-informed neural networks. Adv. Neural Inf. Process. Syst. 34, 26548–26560.

Lawal, Z.K., Yassin, H., Lai, D.T.C., Che Idris, A., 2022. Physics-informed neural network (PINN) evolution and beyond: a systematic literature review and bibliometric analysis. Big Data and Cognitive Computing 6 (4), 140.

Linse, Dennis J., Stengel, Robert F., 1993. Identification of aerodynamic coefficients using computational neural networks. J. Guid. Control Dynam. 16 (6), 1018–1025.

Meng, X., Li, Z., Zhang, D., Karniadakis, G.E., 2020. PPINN: Parareal physics-informed neural network for time-dependent PDEs. Comput. Methods Appl. Mech. Eng. 370, 113250.

MFN Lib., NVIDIA Corporation. Modified Fourier network. Available: https://docs.nvidia.com/deeplearning/modulus/modulus-v2209/user_guide/theory/architectures.html#modified-fourier-network. (Accessed 3 May 2024).

Naderibeni, M., Reinders, M.J., Wu, L., Tax, D.M., 2024. Learning Solutions of Parametric Navier-Stokes with Physics-Informed Neural Networks arXiv preprint arXiv:2402.03153.

NVIDIA Modulus lib., NVIDIA Corporation, "NVIDIA Modulus". Available: https://developer.nvidia.com/modulus (Accessed on May 3, 2024).

OpenFoam soft., OpenCFD, "OpenFoam". [computer software]. Available: https://www.openfoam.com/(Accessed on May 3, 2024).

Parish, Eric J., Duraisamy, Karthik, 2016. A paradigm for data-driven predictive modeling using field inversion and machine learning. J. Comput. Phys. 305, 758–774. https://doi.org/10.1016/j.jcp.2015.11.012. ISSN 0021-9991. https://www.sciencedirect.com/science/article/pii/S0021999115007524.

Pinelli, L., Marconcini, M., Pacciani, R., Bake, F., Knobloch, K., Gaetani, P., Persico, G., 2022. Effect of clocking on entropy noise generation within an aeronautical high pressure turbine stage. J. Sound Vib. 529, 116900.

Rahaman, Nasim, Baratin, Aristide, Arpit, Devansh, Draxler, Felix, Lin, Min, Hamprecht, Fred A., Bengio, Yoshua, Courville, Aaron, 2019. On the spectral bias of neural networks. ICML.

Raissi, M., Perdikaris, P., Karniadakis, G.E., 2019. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J. Comput. Phys.

SeqSolver lib., NVIDIA Corporation. NVIDIA Modulus sequential solver. Available: https://github.com/NVIDIA/modulus-sym/blob/main/modulus/sym/solver/sequential.py#L30. (Accessed 29 July 2024).

Sun, L., Gao, H., Pan, S., Wang, J.X., 2020. Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data. Comput. Methods Appl. Mech. Eng. 361, 112732.

Szabó, B., Babuška, I., 2021. Finite Element Analysis: Method, Verification and Validation.

Tamil Thendral, M., Ganesh Babu, T.R., Chandrasekar, A., Cao, Y., 2022. Synchronization of Markovian jump neural networks for sampled data control systems with additive delay components: analysis of image encryption technique. Math. Methods Appl. Sci.

Tancik, M., Srinivasan, P., Mildenhall, B., Fridovich-Keil, S., Raghavan, N., Singhal, U., Ramamoorthi, R., Barron, J., Ng, R., 2020. Fourier features let networks learn high frequency functions in low dimensional domains. Adv. Neural Inf. Process. Syst. 33, 7537–7547.

Wang, J.X., Wu, J.L., Xiao, H., 2017. Physics-informed machine learning approach for reconstructing Reynolds stress modeling discrepancies based on DNS data. Phys. Rev. Fluids 2, 034603. https://doi.org/10.1103/PhysRevFluids.2.034603.

Wang, S., Sankaran, S., Wang, H., Perdikaris, P., 2023. An Expert's Guide to Training Physics-Informed Neural Networks arXiv preprint arXiv:2308.08468.

Wang, H., Lu, L., Song, S., Huang, G., 2023. Learning Specialized Activation Functions for Physics-Informed Neural Networks arXiv preprint arXiv:2308.04073.

Wight, C.L., Zhao, J., 2020. Solving Allen-Cahn and Cahn-Hilliard Equations Using the Adaptive Physics Informed Neural Networks arXiv preprint arXiv:2007.04542.

Xiang, Z., Peng, W., Zheng, X., Zhao, X., Yao, W., 2021. Self-adaptive loss balanced Physics-informed neural networks for the incompressible Navier-Stokes equations. arXiv preprint arXiv:2104.06217.

Xu, S., Sun, Z., Huang, R., Guo, D., Yang, G., Ju, S., 2023. A practical approach to flow field reconstruction with sparse or incomplete data through physics informed neural network. Acta Mech. Sin. 39 (3), 322302.

Zienkiewicz, Olek C., Taylor, Robert L., Zhu, Jian Z., 2005. The Finite Element Method: its Basis and Fundamentals. Elsevier.